MobileDeepfake: A Lightweight Cascaded Deepfake Detection System for Mobile Deployment

Wing Yam Ho
The Hong Kong Polytechnic University
23041062G

December 19, 2025

Abstract

Deepfakes threaten media integrity and platform trust, yet many detectors that perform well on curated benchmarks degrade sharply under distribution shift. For practical use, we target on-device detection where latency, footprint, and privacy constraints are strict, and where missed fakes (false negatives) carry disproportionate risk. We present MobileDeepfake, a two-inference cascade: a lightweight MobileNetV4 makes high-confidence decisions quickly, while uncertain samples are escalated to a higher-accuracy EfficientNetV2 expert. The system is trained across multiple datasets and tuned with a cost-aware threshold grid search that explicitly minimizes false negatives under an escalation-rate budget, and we export compact quantized artifacts for mobile integration.

Our pipeline is reproducible endtoend: data manifests and loaders enable multi-source training; optional hard-example mining ablations concentrate the Stage 2 expert on ambiguous cases, while the default released training script uses uniform sampling and corresponds to our main reported results; and a tuning/evaluation suite generates the tables and figures used in the paper. On combined validation, the EfficientNetV2 expert achieves high AUC. The tuned cascade attains high AUC while driving FNR to low, exposing an interpretable recall-vs-compute trade-off. We assess robustness under common perturbations (JPEG, noise, blur, brightness) and apply temperature scaling for probability calibration when thresholds transfer across datasets. Cross-dataset evaluation on Deepfake-Eval-2024 highlights the generalization gap typical of in-the-wild media, reinforcing the need for calibration and domain-aware tuning.

We focus on deployability rather than proposing a new backbone: the contribution is a practical, auditable methodology that couples a two-stage cascade with explicit threshold controls, calibration, and a mobile export path. Released artifacts and scripts allow results to be reproduced and extended (e.g., swapping the expert, adding distillation/QAT, or adopting per-family thresholding for robustness). This work provides a template for balancing recall, efficiency, and maintainability in mobile deepfake detection.

1 Introduction

Deepfakes threaten trust, safety, and platform integrity. Synthetic face-swap videos and images, with generative models quickly becoming general purpose and easy to use, are increasingly weaponized for harassment, disinformation and misleading narratives, scams and frauds, intimate imagery without consent. Politically motivated deepfakes have been implicated in interfering with real-world elections; celebrity deepfakes are being weaponized in scams; synthetic identity fraud has cost institutions at least tens of millions of pounds. While conventional image manipulations

disrupt the core tenets of authenticity, deepfakes produced by GANs or diffusion models (or increasingly complex reenactment systems) can ultimately reach photorealism both for human observers and for naive automated detectors.

Simultaneously, media consumption is becoming more heavily biased toward mobile devices—over 60% of internet traffic is on phones, and most of TikTok & Instagram's billions of users consume it through a mobile app, as does WhatsApp. Practical deepfake countermeasures must operate on underpowered phones/tablets, in a world of poor network access and strict privacy requirements. In many deployments, particularly privacy or security-focused ones (as well as offline tools), no media leaves the device except the aggregate logs of detection signals. This on-phone constraint precludes server-side processing as a fallback on bad detections and forces models to be simultaneously accurate, compact, and fast.

Mobile platforms therefore inhabit a fundamental tension between strong detection accuracy and a user-friendly experience. The model must have a high recall (low false negative rate), ensuring harmful content does not evade detection by the model, but at the same time it must also be small enough to not break app size budgets and fast enough to avoid noticeable latency and battery drain. Under distribution shift—where it encounters manipulation techniques, compression pipelines, and content types not seen during training—detectors should still be robust without reverting to expensive fallback models for all inputs. We focus on this on-device deployment setting, and the three primary challenges it entails: (i) cross-dataset generalization in the wild, (ii) low latency and small model footprint on commodity hardware, and (iii) control of false negatives (missed fakes) under operational compute budgets.

Deepfake detection research covers a vast space. Deepfake detection research covers a vast space including spatial, frequency, and temporal cues, multimodality, dataset curation, and evaluation protocols. Recent comprehensive surveys [17, 7] document two longstanding gaps which motivate our work: (i) detectors that perform well on curated benchmarks often severely underperform under distribution shift, and (ii) the majority of evaluation assumes server class hardware as opposed to mobile devices. These gaps imply an opportunity for system designs that consider cross-dataset generalization and mobile constraints as first-class goals rather than an afterthought.

Deepfake Detection as a Problem Space. Deepfake detection is then a binary classification problem of whether given an image or frame contains a real or synthetically manipulated face. While this sounds simple, the class member properties are produced by diverse manipulation techniques (face swapping, face reenactment, attribute editing, full synthesis) while the environmental properties (compression artifacts, lighting, camera sensors, post-processing) also vary widely. Early works used hand-crafted forensic features like head pose inconsistencies [19], blinks of the eyes, JPEG compression artifacts [20], while more modern methods let deep learning automatically extract hierarchical representations directly from pixels or Fourier/frequency spectra producing state of the art accuracies on in-distribution benchmarks [4, 21, 5].

Cross-dataset generalization is a distant goal: models trained on FaceForensics++ or CelebDF suffer AUC drops of 20-30% evaluated on in the wild datasets such as WildDeepfake or Deepfake-Eval-2024. This generalization gap is primarily the result of dataset bias (e.g. training set would overrepresent certain generators/compression settings), domain shift (e.g. social media would apply proprietary compression pipelines) and the rapid evolution of synthesis techniques. Bridging these gaps requires training protocols on multiple datasets exposing models to diverse artifacts, as well as trustworthy cross-dataset protocols that allow out-of-distribution/OOD generalization from studying the robustness of such methods beyond just in-distribution accuracy.

Mobile Deployment Constraints. Most deepfake detection research operates under the assumption of unlimited computation resources—models are trained and evaluated on server GPUs with batch sizes and latencies that are untenable for on-device inference. Many real-world use cases (live video verification, client-side content moderation, edge forensics, etc.) must be deployed on commodity smartphones or embedded devices with limited memory, power budgets, and thermal envelopes. Mobile deployment comes with additional wrinkles: (i) model size must reside within common app bundle size constraints (50–100 MB), (ii) per-frame latency should be amenable to real-time or near-real time inference (e.g., < 200 ms), and (iii) energy draw should maintain battery life over hours-long usage.

Several strands of prior work are relevant to this goal. Deepfake-specific detectors explore spatial, frequency, and physiological cues [7, 8, 9], while lightweight vision models such as MobileNet and EfficientNet families target efficient inference on ARM-class hardware [10, 11, 12, 13]. Separately, early-exit networks and cascaded systems [14, 15, 16] show how to trade compute for accuracy by routing easy samples to shallow branches. However, existing deepfake work rarely combines these ingredients into an end-to-end, auditable system that is (i) trained across multiple datasets, (ii) tuned with explicit risk and cost metrics, and (iii) packaged as a reproducible mobile deployment template. Most studies either evaluate a single heavy detector on server-class GPUs or treat mobile optimization as a brief post-processing step.

In this work we adopt a system-level perspective and design a cascaded detector tailored for on-device deployment. Our goal is to expose explicit knobs over recall, escalation rate, and compute while keeping the training and evaluation pipeline reproducible from raw manifests to mobile artifacts. Against this backdrop, we ask:

- *How can we design a deepfake detector that provides high recall under distribution shift while remaining deployable on resource-constrained mobile devices?*
- *Can a cascaded architecture with explicit thresholds offer interpretable trade-offs between compute and risk, without sacrificing reproducibility?*
- *What minimal set of scripts and artifacts is needed so that practitioners can audit, reproduce, and extend the full pipeline?*
- *How should we evaluate and report robustness—for example under compression, noise, and blur—and cross-dataset generalization in a way that directly informs deployment decisions?*

Existing works on mobile vision and early exits provide building blocks (lightweight backbones, quantization, cascades), but rarely integrate them into an end-to-end, auditable system tailored to deepfake detection with multi-dataset training and out-of-distribution (OOD) evaluation.

Contributions. We design **MobileDeepfake**, a cascaded detection system engineered for mobile deployment and rigorous evaluation. Our contributions are:

- **A cost-aware, two-stage cascade with explicit risk controls.** A lightweight MobileNetV4 Stage 1 acts as a fast filter; ambiguous samples are escalated to an EfficientNetV2 expert (Stage 2), with an optional Stage 3 LightGBM meta-model. We define Stage 1 leakage (the fraction of fakes incorrectly accepted at Stage 1) and Stage 2 escalation rate as primary system metrics, and use a simple two-threshold routing scheme coupled with a cost-sensitive grid search (formalised in Section 5.1) to expose interpretable trade-offs between false negative rate (FNR) and compute.
- **Multi-dataset training and cross-dataset evaluation, including Deepfake-Eval-2024.** We train on four academic datasets and hold out Deepfake-Eval-2024 for OOD validation and

test, quantifying performance gaps and highlighting practical failure modes under in-the-wild distribution shift. Our experiments report both single-model baselines and cascaded results, with calibrated vs. uncalibrated comparisons when transferring thresholds across datasets.

- **An end-to-end, scriptable pipeline with hard-example mining, calibration, and robustness analysis.** The system includes data preprocessing, manifest generation, hard-sample mining, expert training, meta-model fitting, and threshold tuning, with scripts that generate the tables and figures used in this paper. Robustness sweeps over common corruptions (JPEG compression, noise, blur, brightness) and a Deepfake-Eval-2024 evaluation stage are integrated into the same pipeline.
- **A mobile-oriented export and integration template.** We provide a mobile optimization stage that applies quantization and knowledge distillation to shrink models, and a deployment stage that exports compact TorchScript artifacts (and ONNX bundles when needed) together with thresholds/temperature in a metadata file suitable for Android integration. While our focus is on methodology rather than a new backbone, the implementation illustrates how to turn deepfake research pipelines into deployable, auditable systems.

Concretely, MobileDeepfake follows a train $\rightarrow$ tune $\rightarrow$ deploy workflow across six stages. The core online inference in our final system is a two-stage cascade: Stage 1 trains a lightweight MobileNetV4 filter for fast screening; Stage 2 builds an EfficientNetV2 expert and produces per-sample logits/features. Stage 3 optionally fits a LightGBM meta-model on K-fold meta-datasets to study fusion but is not part of the default deployment. Stage 4 grid-searches routing thresholds to minimize false negatives under an escalation-rate budget and logs Stage 1 leakage alongside Stage 2 usage. Stage 5 conducts cross-dataset and robustness evaluation (including Deepfake-Eval-2024), and Stage 6 exports quantized artifacts for on-device integration.

We focus on deployability rather than proposing a new backbone: the contribution is a practical, auditable methodology that couples a two-stage cascade with explicit threshold controls, calibration, robustness analysis, and a mobile export path. We do explore stronger hybrid CNN-Transformer experts such as GenConViT in our Stage 2 ensemble, but find that they offer limited and unstable gains relative to EfficientNetV2-B3 under our constraints, so we relegate them to an appendix and base the default cascade on MobileNetV4 + EfficientNetV2. Subsequent sections detail the datasets and task, the cascaded method and optimization objectives, the experimental setup, and empirical results on in-distribution and out-of-distribution data, followed by a discussion of limitations and deployment considerations.

2 Background

Deepfake detection research spans spatial, frequency, temporal, and multimodal cues, as well as dataset curation and evaluation protocols. Recent comprehensive surveys [17, 7, 18] highlight two persistent gaps that motivate our work: (i) detectors that perform well on curated benchmarks often degrade sharply under distribution shift, and (ii) most evaluations assume server-class hardware rather than mobile devices. These gaps underscore the need for system designs that treat cross-dataset generalization and mobile constraints as first-class objectives rather than afterthoughts.

Deepfake Detection as a Problem Space. Deepfake detection is fundamentally a binary classification task: given an image or video frame, determine whether it depicts a real or synthetically manipulated face. The challenge lies in the diversity of manipulation techniques (face swapping, reenactment, attribute editing, full synthesis) and the variability of real-world conditions (compression artifacts, lighting, camera sensors, post-processing). Early approaches relied

on handcrafted forensic features such as head pose inconsistencies [19], eye blinking patterns, or JPEG compression artifacts [20]. Modern methods leverage deep learning to extract hierarchical representations directly from pixels or frequency spectra, achieving high accuracy on in-distribution benchmarks [4, 21, 5].

However, cross-dataset generalization remains elusive: models trained on FaceForensics++ or CelebDF can exhibit AUC drops of 20–30% when evaluated on in-the-wild datasets like WildDeepfake [6] or Deepfake-Eval-2024 [1]. This performance gap stems from dataset bias (e.g., training sets overrepresent specific generators or compression settings), domain shift (e.g., social media platforms apply proprietary compression pipelines), and the rapid evolution of synthesis methods. Addressing these gaps requires multi-dataset training protocols that expose models to diverse artifacts, as well as rigorous cross-dataset and out-of-distribution (OOD) evaluation protocols that measure robustness beyond in-distribution accuracy.

Mobile Deployment Constraints. Most deepfake detection research assumes unlimited computational resources—models are evaluated on server GPUs with batch sizes and latencies that are impractical for on-device inference. Yet many real-world applications (e.g., live video verification, client-side content moderation, edge forensics) require deployment on commodity smartphones or embedded devices with constrained memory, power budgets, and thermal envelopes. Mobile deployment introduces additional challenges: (i) model size must fit within typical app bundles (50–100 MB), (ii) per-frame latency must support real-time or near-real-time inference (e.g., <200 ms), and (iii) energy consumption must preserve battery life across extended usage. These constraints motivate the use of lightweight architectures, model compression techniques, and adaptive inference strategies that balance accuracy and efficiency.

Core Detection Approaches. Modern deepfake detectors primarily use convolutional neural networks (CNNs) or vision transformers (ViTs) as feature extractors. CNN-based approaches [4, 22, 11] leverage hierarchical convolutional layers to capture spatial artifacts such as blending boundaries, facial inconsistencies, and generative noise patterns. The MobileNet [23, 24, 25, 10] and EfficientNet [22, 11] families exemplify architectures optimized for mobile and resource-constrained settings through depthwise separable convolutions and compound scaling.

Frequency-domain methods [8, 9, 26] exploit the observation that GAN-generated images often exhibit distinctive patterns in their Fourier or DCT spectra, providing complementary cues that survive spatial-domain compression. Recent work demonstrates that combining spatial and frequency features improves robustness to JPEG compression and other post-processing operations [27, 28].

Vision transformers [2, 3] offer global receptive fields that can capture long-range dependencies and subtle manipulation cues across entire frames. Hybrid CNN-Transformer architectures [29, 30, 31] aim to combine the inductive biases of CNNs (e.g., translation equivariance) with the modeling capacity of transformers, though at the cost of increased computational complexity.

Temporal modeling via 3D CNNs [32, 33], LSTMs [34, 35], or video transformers [36] extends frame-level detection to video sequences, enabling the use of inter-frame consistency and motion artifacts as additional forensic signals. However, temporal approaches typically require multi-frame context windows and are challenging to deploy on mobile devices due to memory and latency constraints.

Model Compression and Efficiency. To bridge the gap between research models and deployable systems, practitioners rely on three primary compression techniques:

- **Pruning** [37, 38, 39]: Removing redundant weights or channels to reduce model size and FLOPs. Structured pruning (removing entire filters) offers better hardware efficiency than unstructured (weight-level) pruning.
- **Quantization** [40, 41, 42]: Representing weights and activations with lower-precision data types (e.g., INT8 instead of FP32). Post-training quantization (PTQ) is fast but limited to 8-bit precision, while quantization-aware training (QAT) enables more aggressive compression (e.g., INT4) at the cost of retraining.
- **Knowledge distillation** [43, 44, 45]: Transferring knowledge from a large teacher model to a compact student model by matching soft probability distributions or intermediate feature representations.

Modern deep learning frameworks (PyTorch, TensorFlow, ONNX Runtime) provide end-to-end toolchains for converting research models into mobile-optimized INT8 variants and exporting them to Android (via ONNX, TFLite) or iOS (via CoreML) runtimes.

Cascade and Adaptive Inference. Cascade systems and early-exit networks [14, 15, 46] offer an alternative efficiency strategy: route easy samples through lightweight classifiers and escalate hard ones to more expensive experts. Classical examples include Viola-Jones face detection [47], which uses a cascade of progressively complex classifiers to reject non-faces early. In deep learning, BranchyNet [14] attaches shallow classifiers to intermediate layers, enabling early termination for confident predictions. Recent work on adaptive computation time (ACT) [48] and dynamic depth networks [49] learns input-dependent routing policies.

However, most cascade work focuses on maximizing accuracy or minimizing FLOPs on fixed test distributions, without exposing interpretable operational metrics (e.g., escalation rate, Stage 1 leakage rate) that platform operators need to tune cost–quality trade-offs. Furthermore, cascade detectors for deepfake detection remain rare in the literature, and seldom report performance under distribution shift.

Calibration and Distribution Shift. Confidence calibration via temperature scaling [50, 51] aligns predicted probabilities with empirical frequencies, improving reliability diagrams and expected calibration error (ECE). Well-calibrated models are particularly valuable for cascade routing: thresholds set on calibrated probabilities better reflect true uncertainty and generalize more robustly to OOD data. However, calibration alone does not eliminate performance gaps under domain shift [7]; a well-calibrated model with 75% accuracy on one distribution may still exhibit poor calibration if its true accuracy on another distribution is 60%. Addressing cross-dataset gaps fundamentally requires domain adaptation techniques [52, 53, 54], diverse training data, and transparent reporting of OOD performance.

Our Approach. Given these challenges, we propose a two-stage cascade system for mobile deepfake detection that explicitly optimizes for both detection quality and deployment efficiency. Our Stage 1 filter uses MobileNetV4 [10]—a lightweight CNN optimized for mobile hardware—to perform fast frame-level screening. Samples with highly confident Stage 1 predictions (very likely real or very likely fake) receive immediate labels, while ambiguous cases escalate to a Stage 2 expert (EfficientNetV2-B3 [11] or optional GenConViT [31]) for high-fidelity classification. We formulate threshold selection as a cost-sensitive optimization problem: minimize false negative rate (FNR) subject to a bounded Stage 2 escalation budget, exposing interpretable trade-offs between recall and compute cost.

We train both stages on a combined multi-dataset manifest (CelebDF-v2, FaceForensics++, DFDC, DeeperForensics-1.0) to promote cross-dataset generalization, experiment with hard example mining to focus Stage 2 capacity on cases where Stage 1 is uncertain, and validate the cascade on the in-the-wild Deepfake-Eval-2024 benchmark [1] to measure OOD performance. In our main results and public training script, we use the simpler standard configuration with uniform sampling and treat hard-example mining as an ablation. For mobile deployment, we apply post-training quantization (INT8) and export calibrated ONNX models with fixed thresholds, achieving sub-200 ms latency on a Xiaomi 13 smartphone (Snapdragon 8 Gen 2).

The remainder of this paper proceeds as follows: Section 3 surveys related work in detail, positioning our contributions within the broader landscape of deepfake detection, cascade architectures, and mobile deployment. Section 4 describes our datasets and task formulation. Section 5 presents our six-stage methodology, including formal problem definitions, cascade optimization, and hard example mining. Sections 6 and 7 detail experimental setup and results on in-distribution validation and OOD benchmarks. Section 8 reports ablation studies, and Section 9 discusses mobile deployment specifics (ONNX export, on-device latency, accuracy). Finally, Section 10 discusses limitations and future work, and Section 11 concludes.

3 Related Work

Detection Architectures

CNN-Based Approaches

On early deepfake benchmarks such as FaceForensics++ and DFDC, Xception-based CNNs became de facto baselines [4, 21, 5]. EfficientNet [22] and its successor EfficientNetV2 [11] achieved strong performance through compound scaling that optimizes depth, width, and resolution jointly. Recent studies [34, 55] demonstrate that even lightweight architectures like MobileNetV3 [25] can achieve competitive detection accuracy when properly fine-tuned. ResNet-family models [56], including ResNeXt variants, have been widely adopted for spatial feature extraction in deepfake detection. Informed by these CNN architectures and mobile efficiency considerations, we adopt MobileNetV4 for our Stage 1 filter and EfficientNetV2 for our Stage 2 expert, balancing detection accuracy with on-device latency constraints.

Frequency-Domain Methods

Frequency analysis has emerged as a powerful approach for detecting deepfake artifacts that survive spatial-domain compression. Durall et al. [8] demonstrated that GAN-generated images exhibit distinctive frequency patterns in their spectral distributions. Qian et al. [9] proposed adaptive frequency filtering to emphasize manipulation cues in the frequency domain. More recent work [26, 28] leverages Discrete Cosine Transform (DCT) and Fast Fourier Transform (FFT) to extract high-frequency artifacts as forensic fingerprints. FreqNet [26], introduced at AAAI 2024, applies convolutional layers to both phase and amplitude spectra between FFT and inverse FFT operations, achieving state-of-the-art generalization (+9.8% improvement) across 17 GANs. Studies using Discrete Fourier Transform [57] report classification accuracies exceeding 99% when frequency-domain features are combined with traditional ML classifiers. Wang et al. [27] proposed Adaptive Frequency learning (AFD) that dynamically adjusts frequency emphasis during training to improve robustness under compression. While we focus on spatial-domain CNN architectures for simplicity and inference speed, frequency-domain extensions remain compatible with our cascade framework and could provide complementary robustness under post-processing.

Vision Transformers and Hybrid Architectures

Vision Transformers (ViTs) have recently been adapted to deepfake detection with promising results. Vanilla ViT [2] and hierarchical variants such as Swin Transformer [3] provide global receptive fields that can capture subtle manipulation artifacts across the entire image. A comprehensive 2024 survey [58] categorizes 14 ViT-based deepfake detection models into standalone, sequential (e.g., MARLIN, ISTVT), and parallel (e.g., M2TR) architectures. UIA-ViT [59] achieved 99.33% AUC on FaceForensics++ by incorporating uncertainty-informed attention mechanisms.

Hybrid CNN-Transformer architectures combine the inductive biases of CNNs with the global modeling capacity of transformers. Coccomini et al. [29] combined EfficientNet and vision transformers for video deepfake detection in 2022, while Khan and Dang-Nguyen [30] proposed a Hybrid Transformer Network that fuses convolutional features with transformer encodings. GenConViT [31], which we explore in our Stage 2 expert ensemble, represents a generative convolutional vision transformer that operates in dual modes (hybrid and pretrained). Under our multi-dataset training and mobile deployment constraints, GenConViT matched but did not consistently surpass EfficientNetV2-B3 and exhibited higher training instability, so we treat it as an exploratory component summarised in the appendix rather than part of the default deployed cascade. CoAtNet-based ensembles [60] leverage self-attention mechanisms in bagging frameworks for manipulated face detection. Das et al. [61] introduced masked autoencoding with spatiotemporal transformers for enhanced video forgery detection.

Attention Mechanisms and Multi-Scale Analysis

Attention mechanisms have proven effective in focusing on manipulation artifacts. Multi-attention networks [62] combining region-independent loss and attention-guided data augmentation achieved state-of-the-art performance across multiple datasets. ReLAF-Net [63] employs restricted self-attention on deep CNN features to learn both fine-grained local relationships and global dependencies. MobileNetV3 augmented with self-attention layers [64] demonstrated that lightweight architectures can benefit significantly from attention mechanisms while maintaining deployment efficiency. Spatiotemporal attention mechanisms [65, 35] have been integrated with ConvLSTM to capture and enhance frame-to-frame correlations in video sequences.

Temporal and Video-Level Detection

Beyond spatial architectures, temporal modeling helps handle partial manipulations and contextual cues in video [6, 66]. 3D CNNs [32] extend 2D convolutional filters to spatiotemporal dimensions, enabling direct learning from video clips. The I3D (Inflated 3D) model [33] modifies 2D Inception networks by inflating convolutional and pooling kernels to 3D. Hybrid CNN-LSTM architectures [34, 67] use CNNs for spatial feature extraction at the frame level, then feed sequential features into LSTMs for temporal modeling. Xception-LSTM with ConvLSTM [35] incorporates spatiotemporal attention before dimension reduction to preserve frame structure information. EfficientNet-BiLSTM [68] and similar approaches extract spatial information via EfficientNet and acquire temporal information through bidirectional LSTMs. TimeSformer [36] and other video transformers provide pure transformer-based alternatives for spatiotemporal reasoning.

Parameter-efficient adapters (e.g., DeepFake-Adapter [69]) further improve transfer to in-the-wild data by tuning small sets of adapter parameters on top of a frozen backbone, reducing computational cost while maintaining generalization. Other lines exploit physiological signals (e.g., eye blinking [19], head motion) and camera fingerprints [20], but typically rely on specialized preprocessing pipelines. In this work we focus on frame-level classification for reproducibility and mobile

latency, using generic image backbones (MobileNetV4 and EfficientNetV2) and optional GenConViT experts. Our pipeline remains compatible with temporal post-processing when budgets allow, but deliberately keeps the core inference path simple enough for on-device deployment.

Lightweight Models and Compression

Efficient Architecture Design

Mobile vision emphasizes efficient backbones tailored to on-device constraints. The MobileNet family [23, 24, 25] pioneered depthwise separable convolutions to reduce computational cost while maintaining accuracy. MobileNetV3 [25] incorporated hardware-aware NAS and squeeze-and-excitation modules to optimize for mobile CPUs and GPUs. The latest MobileNetV4 [10] further scales up mobile CNNs with hybrid architectures and improved training recipes. EfficientNet [22] and its successor EfficientNetV2 [11] deliver strong accuracy under tight FLOPs budgets through compound scaling that jointly optimizes network depth, width, and resolution. ShuffleNet [12, 13] and SqueezeNet [70] provide alternative efficient designs that prioritize inference speed on resource-constrained devices.

Hardware-aware neural architecture search (NAS) [71] and once-for-all training [72] enable per-device specialization from a single supernet, allowing practitioners to extract task-and-device-specific sub-networks without retraining from scratch. Recent real-time deepfake detection systems [73, 55] demonstrate that lightweight models such as MobileNetV3 and binary neural networks can achieve competitive performance when combined with attention mechanisms and proper fine-tuning strategies.

Model Compression Techniques

Pruning and sparsity methods reduce model size by removing redundant weights and connections. Structured pruning removes entire channels or filters, while unstructured pruning targets individual weights [39]. Han et al. [37, 38] demonstrated that deep networks can be compressed by 10-50$\times$ through iterative pruning, quantization, and Huffman coding. The Lottery Ticket Hypothesis [74] suggests that dense networks contain sparse sub-networks that can achieve comparable accuracy when trained from the right initialization. Recent surveys [39] provide comprehensive comparisons of pruning methods and their impact on inference efficiency.

Knowledge distillation [43] condenses a large teacher model to a compact student model by minimizing the differences between soft probability distributions or intermediate feature representations. FitNets [44] extends the matching hint layers to intermediate depths, and Born-Again Networks [75] use multi-generation distillation where students can even beat their teachers! The recent survey [45] synthesizes all these trends, observing distillation is more effective in conjunction with pruning and quantization. Work into knowledge distillation for edge deployment [76, 77] quantify the trade-off, and find distilled models in these settings can achieve 70% compression for only a small accuracy drop of 1-3% in exchange for being able to be deployed onto fog/edge devices on applications such as automatic disease diagnosis (sensors/diagnostics), or on-device deepfake detection!

Quantization for Mobile Deployment

Quantization is a key lever for efficient inference, both in terms of model size and inference cost, by reducing the precision used to represent weights and activations. Post-training quantization (PTQ) [40, 41] directly converts learned final FP32 models into lower precision INT8 or similar.

The process is fast and lightweight (i.e., there is no need for retraining for speedup). In quantization-aware training (QAT) [41, 78], quantization is modeled when training the model, allowing reduced precision (e.g., INT4) while achieving competitive accuracy. This method requires access to training data, introducing a heavier computational burden and preventing its use in all cases. A survey in a white paper [42] walks through the variations of quantization schemes, finding that PTQ is often good enough to quantize weights to 8-bits while "maintaining near baseline FP32 model accuracy." QAT allows for more aggressive sub-8-bit compression.

Mixed-precision and per-channel quantization schemes [78, 41] provide competitive accuracy at low bitwidths by allowing different layers or channels to use different quantization scales. Recent work on quantizing large language models [79, 80] and vision transformers demonstrates that quantization remains effective even for modern architectures. For deepfake detection, quantization enables deployment on commodity mobile devices with minimal accuracy loss, and modern deep learning frameworks provide end-to-end toolchains for converting research models into compact INT8 variants and exporting them to mobile platforms.

Our Stage 1 uses MobileNetV4 as a fast filter, and Stage 2 adopts EfficientNetV2 and optional GenConViT experts [11, 31]. Stage 4 builds on QAT + knowledge distillation in a teacher–student setup and compares FP32 vs. INT8 models in terms of accuracy, size, and latency, while Stage 6 exports TorchScript and ONNX artifacts suitable for mobile integration. We emphasize measurable footprint and latency on device: model size, on-device escalation rate, and duty cycle jointly determine energy and user experience, so we report escalation alongside accuracy when comparing options.

Cascade and Adaptive Inference

Early-Exit Networks and Dynamic Depth

Early-exit and cascaded systems trade accuracy for efficiency by routing easy samples to fast exits and escalating hard ones. BranchyNet [14] pioneered this approach by attaching shallow classifiers to intermediate layers of deep networks, enabling early termination for confident predictions. Multi-Scale Dense Networks (MSDNet) [15] extend this idea by learning feature representations at multiple scales, allowing classifiers at different depths to leverage complementary information. Shallow-Deep Networks [16] address network overthinking by identifying when deeper layers degrade performance on easy samples.

A comprehensive 2024 survey on early-exit deep neural networks [46] categorizes methods into static (BranchyNet, MSDNet) and adaptive (ACT, SkipNet) approaches. Adaptive Computation Time (ACT) [48] for RNNs and SkipNet [49] for CNNs learn dynamic routing policies that adapt network depth to input complexity. Recent work on window-based early-exit cascades [81] incorporates uncertainty estimation for improved reliability. QuickNets [82] introduced a novel cascaded training algorithm where each successive layer is trained only on samples misclassified by previous layers, preventing overconfidence and improving calibration.

Routing Policies and Threshold Selection

Confidence-, entropy-, and margin-based routing are common strategies for determining when to exit early. Hendrycks and Gimpel [83] established maximum softmax probability as a simple baseline for detecting misclassified examples and out-of-distribution samples. Cost-sensitive learning frameworks [84] formalize the trade-off between classification errors and computational costs, enabling threshold selection that minimizes expected cost rather than maximizing accuracy alone. Recent work increasingly frames threshold selection as a multi-objective optimization problem that

balances error rates (false positives, false negatives) against compute budgets (FLOPs, latency, energy) [85, 86].

Cascaded systems for object detection [47] and classification have been deployed successfully in production environments, but typically employ hand-tuned thresholds rather than learned policies. Bayesian optimization [87] and grid search remain popular for threshold tuning when the search space is small. For multi-stage systems, joint optimization of all thresholds is often intractable, motivating greedy stage-wise tuning or heuristic policies.

Cascades in Deepfake Detection

In deepfake detection, cascades are less explored than single-model detectors, and are rarely paired with explicit system-level metrics such as Stage 1 leakage rate and Stage 2 escalation rate. Most prior work reports AUC/F1 on a fixed test set without exposing the operational trade-offs platform operators care about. Classical cascade detectors like Viola-Jones [47] for face detection demonstrated the effectiveness of multi-stage architectures, but deepfake detection has largely focused on maximizing single-model accuracy rather than optimizing inference efficiency.

Our system follows the early-exit lineage but makes the design explicitly cost-aware: a lightweight Stage 1 filter (MobileNetV4) sits in front of a stronger Stage 2 expert (EfficientNetV2 + GenConViT) and optional Stage 3 meta-model (LightGBM), and a two-threshold routing scheme defines a tunable "uncertainty band" for escalation. Stage 4 performs grid search over low/high thresholds using cached logits, optimizing for low false negative rate under an escalation-rate budget (typically 1–5%). The resulting operating points are reported with both FNR and Stage 2 escalation rate, exposing interpretable trade-offs between risk and compute. For deployment, we keep the runtime policy intentionally simple: thresholds are tuned offline via grid search and then fixed on device for stability, avoiding the complexity of learned routing policies that may drift under distribution shift.

Generalization and Robustness

Cross-Dataset Generalization

Cross-dataset generalization is a core challenge in deepfake detection: large AUC drops (often 20–30%) are observed when models trained on curated datasets like FaceForensics++ are evaluated on in-the-wild media [6, 7]. State-of-the-art models achieve 96–99% AUC on in-dataset tests but degrade to 72–78% on cross-dataset evaluations [88]. This performance gap stems from dataset bias, compression artifacts, and differences in manipulation techniques across generators.

Domain Adaptive Batch Normalization (DABN) [53] mitigates distribution gaps by utilizing test-set statistics to restore batch normalization layer effectiveness, achieving nearly 20% accuracy improvement on CelebDF under black-box settings. Recent work on unsupervised domain adaptation [54] transfers deepfake detection knowledge from labeled source domains to unlabeled target domains using Domain Distance Optimization (DDO) modules that align inter-domain and intra-domain feature distances. Spatial-frequency collaborative learning methods [89] outperform frequency-aware baselines by 9.5% AUC on CelebDF through joint spatial and frequency domain modeling.

Domain adaptation surveys [90, 52] provide comprehensive taxonomies of transfer learning and domain-adversarial approaches. Domain-Adversarial Neural Networks (DANN) [52] learn domain-invariant features through adversarial training between feature extractors and domain classifiers. However, achieving consistent cross-dataset performance remains elusive: fairness-aware methods [91] maintain fairness metrics on some test sets but fail on others, highlighting the difficulty of

generalizing both accuracy and fairness simultaneously. In our work, we train on a combined multi-dataset manifest and evaluate on Deepfake-Eval-2024 as an explicit OOD benchmark to directly measure this generalization gap and report cross-dataset performance honestly.

Robustness to Perturbations and Corruptions

Robustness to common corruptions (JPEG compression, noise, blur, illumination changes) is likewise uneven; even strong detectors can degrade substantially under realistic post-processing [8, 9, 27]. Frequency-domain methods provide one avenue for robustness by emphasizing manipulation cues that survive compression and noise. Adaptive frequency learning [27] dynamically adjusts frequency emphasis during training to improve robustness under JPEG compression. High-frequency enhancement networks [92] for heavily compressed content demonstrate that explicit frequency modeling can recover detection performance even under aggressive compression.

Adversarial robustness is another critical dimension. Carlini and Wagner [93] attacks and Projected Gradient Descent (PGD) [94] perturbations can fool deepfake detectors with imperceptible modifications. While adversarial training [94, 95] improves robustness to ℓ_p-bounded attacks, it often degrades performance on clean samples and does not guarantee robustness to adaptive attacks. Recent comprehensive evaluations [96] assess 8 state-of-the-art defenses against evolving threats (adversarial examples, OOD samples, novel generators), finding that no single method achieves robustness across all threat models.

Benchmarks and Evaluation Protocols

Newer benchmarks explicitly target generalization gaps. Deepfake-Eval-2024 [1] aggregates 1,072 real and 964 fake files from diverse platforms, languages, and compression pipelines as an in-the-wild benchmark. DeepfakeBench [88], presented at NeurIPS 2023, provides a comprehensive evaluation framework supporting 36 detection methods (28 image + 8 video detectors) and introduces the DF40 dataset comprising 40 distinct deepfake techniques including state-of-the-art 2024 generators. Celeb-DF++ [97] extends the original CelebDF benchmark to evaluate 19 classic methods and 5 recent (post-2024) detectors.

Recent surveys [7, 18, 98] emphasize that robustness techniques must be coupled with rigorous cross-dataset evaluation protocols to avoid overfitting to specific artifacts. The surveys categorize methods by feature extraction (spatial, frequency, temporal, multimodal) and network architecture (CNN, Transformer, hybrid), and identify generalization as the primary open challenge for practical deployment.

Calibration and Confidence Estimation

Calibration and thresholding play an important role under distribution shift. Probability calibration techniques such as temperature scaling [50, 51] help align predicted probabilities with empirical frequencies, improving reliability diagrams and expected calibration error (ECE). However, calibration does not eliminate performance gaps on new datasets [7]; a well-calibrated model with 75% accuracy remains poorly calibrated if evaluated on a distribution where its true accuracy is 60%.

Ensemble and meta-learning approaches [99, 100] leverage stacking-based fusion to improve both accuracy and calibration. Naskar et al. [99] combined Xception and EfficientNetV2-B7 features via a meta-learner (MLP), achieving 93% accuracy on CelebDF and 98% on FaceForensics++ through feature ranking and selective fusion. Our Stage 3 meta-model follows this stacking paradigm but uses LightGBM as the meta-learner and incorporates K-fold cross-validation for robustness.

In our evaluation (Stage 5), we therefore combine cross-dataset experiments on Deepfake-Eval-2024 with robustness sweeps over JPEG, noise, motion blur, brightness, downscaling, and gamma perturbations. We report both calibrated cascade results and a Stage 2-only upper bound, and we log Stage 2 escalation rates alongside F1/FNR. This design emphasizes system-level trade-offs: robustness gains are only useful if they do not explode compute or latency on device, and calibration is only useful when interpreted jointly with escalation and leakage statistics rather than as a standalone score.

Deployment and Tooling

Production deployment for mobile requires artifact export, footprint accounting, latency measurement, and minimal telemetry. TorchScript-or ONNX-style runtimes provide stable toolchains for shipping models to Android and iOS, while quantization tooling and surveys [42] make it feasible to convert research models into compact INT8 variants without rewriting training code. Beyond the model itself, practitioners need configuration management (e.g., thresholds, calibration temperatures), logging and monitoring hooks, and a clear separation between offline tuning and online inference.

Our pipeline encompasses these considerations end to end. We export TorchScript artifacts for Stage 1 and Stage 2 as well as a zip of metadata that tracks thresholds, calibration temperatures, etc. ONNX exporters and benchmarking scripts are available for optional runtimes and thorough latency/memory profiling. We also minimise all I/O and allocations on device to reduce jitter as well as maintenance. In this way **MobileDeepfake** occupies the space between research prototype and production system: we care about (i) multi-dataset training and hard-example mining; (ii) cost-sensitive threshold tuning for low false negatives with controlled escalation; and (iii) on-device integration with quantifiable trade-offs. Different from single-model detectors which are in-train black-box parts of the single model, our system exposes dials "visible" by the designer (thresholds, temperature, etc) customized to align with requirements and supported by reproducible scripts and artifacts.

4 Datasets and Task

Task Definition. We frame deepfake detection as binary classification at the frame/image level: *real* (`label=0`) vs. *fake* (`label=1`). For videos, our manifests reference extracted face crops; training and evaluation operate at the frame/image granularity to maximise sample size and label fidelity, while downstream applications may aggregate predictions per video (e.g., majority vote or max probability) if needed. This choice keeps the core task simple and aligns with common evaluation practice on FF++/CelebDF/DFDC [4, 5, 21].

Table 1: Dataset sizes by split (rows counted from manifests).

Split	Dataset	Rows
Train	CelebDF-v2	83,599
	DeeperForensics	844,396
	DFDC	721,946
	FaceForensics++	223,653
Val	CelebDF-v2	17,478
	DeeperForensics	165,854
	DFDC	154,919
	FaceForensics++	50,945
Test	CelebDF-v2	18,037
	DeeperForensics	172,894
	DFDC	154,511
	FaceForensics++	47,430
Val	Deepfake-Eval-2024	451,917
Test	Deepfake-Eval-2024	402,413

Table 2: Per-dataset class balance by split (rows and real/fake counts from balanced manifests).

Split	Dataset	Rows	Real	Fake
Train	CelebDF-v2	83,599	41,674	41,925
	DeeperForensics	844,396	414,512	429,884
	DFDC	721,946	361,229	360,717
	FaceForensics++	223,653	111,161	112,492
Val	CelebDF-v2	17,478	8,765	8,713
	DeeperForensics	165,854	87,807	78,047
	DFDC	154,919	77,350	77,569
	FaceForensics++	50,945	26,351	24,594
Test	CelebDF-v2	18,037	9,118	8,919
	DeeperForensics	172,894	89,253	83,641
	DFDC	154,511	77,109	77,402
	FaceForensics++	47,430	23,502	23,928
Val	Deepfake-Eval-2024	451,917	302,369	149,548
Test	Deepfake-Eval-2024	402,413	216,288	186,125

Datasets. We train on four academic datasets—CelebDF v2, FaceForensics++, DeeperForensics 1.0, and DFDC—and hold out Deepfake-Eval-2024 as an out-of-distribution (OOD) benchmark. CelebDF v2 provides high-quality face-swap videos of celebrities designed to challenge detectors [5]; FaceForensics++ aggregates multiple manipulation methods (DeepFakes, Face2Face, FaceSwap, NeuralTextures, etc.) under controlled conditions [4]; DFDC is a large-scale crowd-sourced dataset with diverse actors and compression pipelines [21]; DeeperForensics 1.0 emphasises real-world perturbations and challenging textures [101]. Deepfake-Eval-2024 adds contemporary in-the-wild content collected from TrueMedia.org and social media platforms in 2024, covering 88 websites and 52 languages [1]. We use Deepfake-Eval-2024 strictly for cross-dataset validation/testing. We adopt a unified 256×256 PNG face-crop specification for all preprocessed datasets, following common practice in MobileNet/EfficientNet-based detectors and balancing fidelity against on-device latency. In addition to these training/evaluation sources, we reference the DF40 dataset introduced in Deep-

fakeBench [88] as a public benchmark that already uses 256×256 PNG face crops; DF40 itself is not included in the training or evaluation splits for the experiments reported in this paper, but its format informs compatibility with newer in-the-wild benchmarks.

Preprocessing Pipeline. All video datasets are preprocessed into face-crop images using a configurable pipeline (cf. [4, 5, 40] for similar face-crop protocols). Frames are sampled at a fixed interval (`frame_interval=10`), faces are detected with MTCNN (minimum face size `min_face_size=20`, thresholds $[0.6, 0.7, 0.7]$), cropped around the bounding box with `bbox_scale=1.3`, and resized to 256×256 pixels. We cap redundancy by keeping at most 50 faces per video (`max_faces_per_video=50`). These parameters are versioned alongside the manifests so that the training set can be regenerated or audited.

The preprocessing stack supports multiple video backends (e.g., OpenCV, TorchVision, Decord) and face detectors (MTCNN, InsightFace, YOLOv8, MediaPipe, RetinaFace), but our final experiments use MTCNN with GPU-accelerated decoding for simplicity and reproducibility. Processed images are organised into train/validation/test splits for each dataset, with separate folders for real and fake faces, and accompanied by CSV manifests that record the necessary metadata for every crop. This approach maintains a simple underlying file layout while making manifests the primary interface for the tasks of training, evaluation, and audit.

Manifest Generation and Structure. After preprocessing, we produce CSV manifests that are the canonical index of all our experiments. Each row of a manifest corresponds to a single face image and includes (at the least) a `image_path`, `label`, `split` and `dataset`; further fields such as `md5`, `valid`, `width`, `height` and `file_size` can be used for integrity checks. In this way Table 1 is derived directly from the manifests. All sampling, hard-example mining, and evaluation components operate on manifest rows rather than raw folders, making it easy to extend the pipeline to new datasets by regenerating manifests.

Split Protocol and Balancing. We adopt a simple and reproducible split protocol. For each dataset, preprocessed faces are partitioned into train/val/test splits according to configuration ratios, preserving official test lists when available (e.g., CelebDF v2 and FF++). We then construct balanced manifests per dataset and split by sampling equal numbers of real/fake faces. For multi-dataset training, we concatenate these manifests and treat each dataset as a source domain; the combined manifest is approximately balanced both by class and by dataset, so Stage 1 and Stage 2 loaders can rely on standard shuffling rather than complex per-batch reweighting. An earlier weighted sampler remains available in the codebase but is not used in our final experiments.

To guard against trivial path-based leakage, the manifest pipeline can optionally sanitise or reject samples with tell-tale substrings (e.g., directory names that encode `/fake/` or `/real/`). These checks are primarily useful when integrating new corpora; for the curated datasets considered here, manifests are derived from neutral directory layouts and the leakage check is disabled by default.

Augmentation and Normalization. Stage 1 employs a compact torchvision transform pipeline that resizes images to 256 px, applies random horizontal flip, colour jitter, small random affine transformations, and a light Gaussian blur, and normalises with ImageNet statistics. Augmentation is symmetric across classes, with a modest probability of colour/geometry perturbations. This design follows contemporary data augmentation practice for image classification while avoiding heavy corruptions that could distort robustness analysis [102, 7]. Stage 2 uses slightly stronger geometric

and colour jitter when beneficial; all stages share the same input resolution and normalisation to simplify deployment and quantization.

OOD Evaluation Protocol. For cross-dataset evaluation, we follow Stage 5 and treat Deepfake-Eval-2024 as a held-out OOD benchmark [1]. We evaluate (i) a calibrated two-stage cascade (Stage 1 + Stage 2 + thresholds tuned on the in-distribution validation set) and (ii) a Stage 2-only upper bound that bypasses Stage 1 and uses the expert on all samples. The tables (Table 8, 9) summarise accuracy, F1, FNR, FPR, and Stage 2 rate on the Deepfake-Eval-2024 val/test splits. The sizable performance gap relative to in-distribution validation underscores the importance of robustness studies and future domain adaptation; in this work we focus on low-friction mitigations such as temperature scaling and threshold sweeps [50, 51], and avoid OOD-specific fine-tuning unless justified by target data.

Dataset Characteristics. Table 3 summarises qualitative properties of each dataset to motivate our protocol design. These traits justify balanced sampling across sources and the use of a dedicated

Table 3: Qualitative characteristics of datasets (informal summary).

Dataset	Generation	Quality	Source	Notes
CelebDF v2	Face-swap	High	YouTube	Celebrities; curated videos
FaceForensics++	Multiple	Mid	Actors	Multiple manipulation families
DFDC	Multiple	Mid–Low	Internet	Large-scale, mixed quality
DeeperForensics 1.0	Multiple	Mid	Lab	Realistic perturbations, hard cases
Deepfake-Eval-2024	Multiple	Mid–Low	Web	In-the-wild, multi-source, 2024 media

OOD set to probe generalization under realistic distribution shift.

Mobile Evaluation Subset. To evaluate on-device performance without requiring the full multi-gigabyte datasets on a smartphone, we construct a compact `mobile_eval` subset. This subset is derived by stratified sampling from the four training datasets (CelebDF-v2, FaceForensics++, DFDC, DeeperForensics 1.0), selecting representative face crops from each source. We sample 1–2 frames per video from a broad sweep of videos, to target ∼150–200 images total.

The resulting subset is then split into two directories: `images_real/` (72 images) and `images_fake/` (80 images); ground-truth labels are determined according to which directory an image belongs to. The Android app can thus determine an image's label by its filesystem path, and compute accuracy without the need for human labeling.

The `mobile_eval` subset is (1) a snapshot that makes it easier and faster to iterate while developing the app without needing to move large datasets back and forth, (2) small enough so that we have reproducibly measured latency and accuracy on-device for measurement, and (3) small enough that we can still do a few full runs of tests in a given testing session. We intentionally reuse processed face crops from our PC pipeline so that on-device and desktop evaluations run on pixel-identical images (besides any JPEG re-encoding due to transfer); all reported on-device metrics in Section 9 refer to this subset.

5 Method

5.1 Problem Formulation

Binary Classification Task. We frame deepfake detection as binary classification over face images. Given an input image $x \in \mathcal{X} = \mathbb{R}^{H \times W \times 3}$ (where $H = W = 256$ in our pipeline), the task is to predict a label $y \in \{0, 1\}$ where $y = 0$ denotes real and $y = 1$ denotes fake. Our training data comprises $\mathcal{D} = \{(x_i, y_i, d_i)\}_{i=1}^{N}$, where d_i indexes the source dataset (CelebDF-v2, FaceForensics++, DFDC, or DeeperForensics-1.0).

Two-Stage Cascade Architecture. Our deployed system consists of two sequential classifiers:

- **Stage 1 (Filter):** A lightweight model $f_1 : \mathcal{X} \to \mathbb{R}$ producing logit $f_1(x)$ and calibrated probability $p_1(x) = \sigma(f_1(x))$, where σ is the sigmoid function.
- **Stage 2 (Expert):** A higher-capacity model $f_2 : \mathcal{X} \to \mathbb{R}$ producing logit $f_2(x)$ and calibrated probability $p_2(x) = \sigma(f_2(x)/t)$, where t is the temperature scaling parameter.

Cascade Decision Policy. The cascade routes samples based on two thresholds $\tau_{\text{low}}, \tau_{\text{high}} \in [0, 1]$ with $\tau_{\text{low}} < \tau_{\text{high}}$:

$$\hat{y}(x) = \begin{cases} 0 & \text{if } p_1(x) < \tau_{\text{low}} \\ 1 & \text{if } p_1(x) > \tau_{\text{high}} \\ \mathbf{1}[p_2(x) > \tau_2] & \text{otherwise} \end{cases} \tag{1}$$

where $\mathbf{1}[\cdot]$ is the indicator function and $\tau_2 \in [0, 1]$ is the Stage 2 decision threshold. Samples falling in the interval $[\tau_{\text{low}}, \tau_{\text{high}}]$ are *escalated* to Stage 2, while those outside this band receive immediate Stage 1 predictions.

Evaluation Metrics. We define standard classification metrics on a test set $\mathcal{D}_{\text{test}}$:

$$\text{FNR}(\tau) = \frac{\sum_{(x,y) \in \mathcal{D}_{\text{test}}: y=1} \mathbf{1}[\hat{y}(x) = 0]}{\sum_{(x,y) \in \mathcal{D}_{\text{test}}: y=1} 1} \tag{2}$$

$$\text{FPR}(\tau) = \frac{\sum_{(x,y) \in \mathcal{D}_{\text{test}}: y=0} \mathbf{1}[\hat{y}(x) = 1]}{\sum_{(x,y) \in \mathcal{D}_{\text{test}}: y=0} 1} \tag{3}$$

The **Stage 2 escalation rate** measures the fraction of samples requiring expert evaluation:

$$\pi_2(\tau_{\text{low}}, \tau_{\text{high}}) = \Pr(\tau_{\text{low}} \leq p_1(x) \leq \tau_{\text{high}}) \tag{4}$$

This metric directly governs computational cost, as Stage 2 inference is significantly more expensive than Stage 1.

Optimization Objective. Our cascade tuning (detailed in subsequent subsections) seeks threshold configurations that minimize FNR subject to an escalation budget:

$$\min_{\tau_{\text{low}}, \tau_{\text{high}}, \tau_2} \text{FNR}(\tau_{\text{low}}, \tau_{\text{high}}, \tau_2) \quad \text{subject to} \quad \pi_2(\tau_{\text{low}}, \tau_{\text{high}}) \leq B \tag{5}$$

where B is a platform-defined budget (e.g., $B = 0.05$ limits escalation to 5% of samples). This formulation captures the core trade-off between detection quality (low FNR) and inference cost (bounded escalation).

Hard Example Mining. To improve Stage 2 performance on ambiguous cases, we define a margin-based ambiguity set:

$$m(x) = |p_1(x) - 0.5|, \quad \mathcal{A}_\delta = \{x : m(x) \leq \delta\} \tag{6}$$

where δ is a margin threshold (e.g., $\delta = 0.4$ corresponds to $p_1(x) \in [0.1, 0.9]$). The hard example set combines misclassifications and ambiguous samples:

$$\mathcal{H} = \{(x, y) \in \mathcal{D} : \mathbf{1}[\hat{y}_1(x) \neq y] \text{ or } x \in \mathcal{A}_\delta\} \tag{7}$$

where $\hat{y}_1(x) = \mathbf{1}[p_1(x) > 0.5]$ is the provisional Stage 1 decision. We use this construction to define hard-example subsets for analysis and ablations. When hard-example mining (HEM) is enabled, we upweight $\mathcal{H}$ at the *sampling* level by oversampling its elements rather than changing the loss function; the released Stage 2 training script defaults to uniform sampling on the combined manifest (HEM disabled) and is used for our main results.

5.2 Pipeline Overview

Following the formal setup above, we now describe our six-stage pipeline designed for both accuracy and deployability. The core decision stack in our final system has two modeling stages (Stage 1–2), followed by tuning (Stage 4), robustness and cross-dataset evaluation (Stage 5), and mobile deployment (Stage 6). Stage 3 explores an optional meta-model that we do *not* deploy by default but keep as a research tool for fusion analysis.

- **Stage 1: MobileNetV4 filter** — fast frame-level screening on device
- ↓ escalate ambiguous cases
- **Stage 2: EfficientNetV2 expert** — high-fidelity logits and embeddings
- ↓ offline threshold and policy search (Stage 4)
- **Stage 4: Cascade tuning** — choose $(\tau_{\text{low}}, \tau_{\text{high}})$ under escalation budget
- ↓ robustness and OOD evaluation (Stage 5)
- **Stage 6: Mobile deployment** — export calibrated models and thresholds to device

Figure 1: System pipeline at a glance. Stage 1 and Stage 2 form the deployed cascade; Stage 3 (LightGBM meta-model) is optional and used for analysis rather than in the default mobile path.

Table 4: Roles of components by stage (high-level summary).

Stage	Component	Backbone / Model	Role
1	Fast filter	MobileNetV4 Hybrid Medium	Frame-level real/fake screening
2	Expert	EfficientNetV2-B3 (optional GenConViT)	High-fidelity logits + embeddings
3 (opt.)	Meta-model	LightGBM	Experimental fusion of expert feature
4	Cascade tuning	Threshold grid search	Optimise FNR under escalation budg
5	Robustness/OOD	Cascade engine + sweeps	Stress tests (perturbations, Deepfake-
6	Deployment	ONNX + metadata	On-device inference bundle for mobile

Stage 1: Lightweight Filter (MobileNetV4). A compact MobileNetV4 variant serves as a fast binary classifier. Concretely, we use timm's `mobilenetv4_hybrid_medium.ix_e550_r256_in1k` and fine-tune it with `BCEWithLogitsLoss` on a unified multi-dataset loader built from pre-balanced manifests (Section 1) with standard `shuffle=True`. Earlier experiments with explicit `WeightedRandomSampler` showed similar behaviour but added complexity, so the final configuration equalizes dataset contributions at the manifest level and keeps the loader simple. A torchvision transform pipeline provides light augmentation (resize to 256 px, random horizontal flip, colour jitter, small random affine perturbations, and Gaussian blur) before normalization. To improve robustness to occasional corrupted images in large video datasets, the data loaders fall back to a zero-valued placeholder frame when an image file cannot be decoded, avoiding training interruptions at the cost of clearly out-of-distribution inputs. We train with AdamW and a cosine annealing scheduler, apply early stopping on validation AUC, and select the best checkpoint for subsequent stages; training runs log curves and metrics for later analysis. The Stage 1 head outputs a single logit per image. A separate calibration step fits a temperature scaling parameter on a validation manifest, producing a JSON file used in cascade tuning and cross-dataset evaluation.

Stage 2: Expert Feature Extractor (EfficientNetV2, optional GenConViT). We train Stage 2 experts to provide higher-fidelity signals for ambiguous cases and to serve as feature extractors for the cascade. In our main results, EfficientNetV2-B3 (`efficientnetv2_b3.in21k_ft_in1k`) serves as the production CNN expert, trained on the combined train manifest with moderately stronger augmentation (RandAugment, Mixup/CutMix at the batch level) and a larger batch size than Stage 1. During Stage 2 experiments we optionally construct a "difficult subset" manifest based on Stage 1 scores (misclassified or low-margin samples) and, in ablation runs with HEM enabled, oversample this subset to focus the expert on corner cases while capping per-dataset counts to prevent drift toward any single source. The default public training script uses uniform sampling on the combined manifest (no HEM) and corresponds to the main EfficientNetV2-B3 results; we treat HEM as an opt-in training variant because it increases implementation complexity and training time, and our primary goal is to keep the released pipeline easy to reproduce and extend. We optimise a focal loss objective with label smoothing to mitigate class imbalance and label noise, matching the hyperparameters reported in Appendix A.1. The expert outputs logits and intermediate embeddings that are cached for cascade tuning (Stage 4) and, when enabled, meta-model experiments (Stage 3).

GenConViT Exploration (optional expert). We additionally explored GenConViT as a transformerstyle expert via a small integration manager that supports hybrid (custom) and pretrained modes with unified interfaces. In AUTO mode, we prefer the pretrained path when available and fall back to hybrid otherwise. Under our configuration and timeline, GenConViT training exhibited instability and domain-specific biases from pretrained weights, and its best validation metrics matched or slightly underperformed EfficientNetV2-B3. As a result, we treat GenConViT as an optional research component rather than part of the default cascade; Appendix A (GenConViT section) summarises the alternativearchitecture experiments and their limitations.

Stage 3: MetaModel Integration (LightGBM). We construct a Kfold metadataset by collecting out-of-fold predictions and features from the experts. For each fold, Stage 2 models are retrained on the fold's training split, and embeddings/logits are extracted on the held-out validation split; this yields unbiased features for meta-learning. Each expert's embeddings are aligned to a unified 256-D space (via projection or zero-padding), and the aligned vectors are concatenated

to form a 512-D meta-feature. In the current implementation, Stage 1 logits are not included but can be added in future variants. A LightGBM classifier [104] is then trained to fuse expert signals, with a modest hyperparameter sweep over learning rate, number of leaves, depth, and L1/L2 regularization. We report the best configuration by validation F1/AUC. Validation always occurs on the full multi-dataset split to avoid overfitting; the meta-model serves as the final decision head for escalated samples when enabled.

Algorithm Summary

For clarity, we summarise the end-to-end training and inference procedures as high-level steps that operate on manifest rows rather than raw folders.

Training Pipeline.

1. Preprocess raw videos into face crops and generate per-dataset train/validation/test manifests with labels and basic metadata.
2. Train the Stage 1 MobileNetV4 filter on a balanced multi-dataset manifest and calibrate it with temperature scaling on a held-out validation set.
3. Train the Stage 2 EfficientNetV2-B3 expert (and any optional research experts) on the same manifest; ablation runs optionally enable hard-example mining by oversampling a hard-subset as defined above, while the default configuration uses uniform sampling. Cache logits and embeddings on validation data.
4. Optionally construct K-fold meta-datasets from out-of-fold expert features and train LightGBM meta-models for fusion.
5. Perform cascade threshold tuning on the combined validation manifest by sweeping $(\tau\_\text{low}, \tau\_\text{high})$ and selecting operating points that trade FNR against the Stage 2 escalation rate under a budget.
6. Run robustness and cross-dataset evaluations, including Deepfake-Eval-2024, under controlled perturbations and store summary metrics and figures.
7. Export calibrated models, thresholds, and lightweight metadata into mobile-ready bundles for deployment on devices.

Inference Pipeline.

1. Apply the Stage 1 filter to each frame; if the calibrated fake probability is confidently below τ_low or above τ_high, return a real/fake decision without escalation.
2. For ambiguous cases with scores in $[\tau\_\text{low}, \tau\_\text{high}]$, run Stage 2 to obtain refined logits (and, when enabled, meta-features).
3. In the default cascade, threshold Stage 2 logits at a fixed value (e.g., 0.5) to obtain the final decision; when the optional meta-model is enabled, feed expert features into it and threshold its output instead.

This summary formalises the textual description above: training proceeds from data preprocessing and manifest construction through Stage 1–3 modelling, threshold tuning, and robustness evaluation. In the default configuration that we deploy and evaluate throughout the paper, online inference uses a fixed two-threshold routing policy between Stage 1 and Stage 2, while the Stage 3 meta-model is kept as an offline analysis component rather than part of the production cascade.

Stage 4: Cascade and Threshold Tuning. We define *global* low/high thresholds on the Stage 1 fake probability $p\_1(x)$ for cost-sensitive offline threshold selection. If $p\_1(x) < \tau\_low$ we predict real; if $p\_1(x) > \tau\_high$ we predict fake; otherwise we escalate to Stage 2 (and, when enabled, optionally to Stage 3) for further analysis. In the configurations reported in Table 14, we restrict attention to the two-threshold cascade (Stage 1 + Stage 2 only) and select operating points such as $(\tau\_low, \tau\_high) = (0.05, 0.55)$ and $(0.03, 0.55)$. We perform an *offline* grid search over (τ_low, τ_high) using cached logits and labels, optimising F1 and minimising FNR under an escalation-rate budget; tuned settings and trade-off plots are logged for auditability and then exported as fixed cascade configurations for deployment. A unified Python cascade prototype in the repository exposes configuration hooks for these thresholds; conservative placeholder values in example configs are superseded by the tuned settings used for all results in this paper when we reproduce tables and figures. We additionally track the Stage 1 leakage rate

$$\pi\_\text{leak} = \Pr\big(p\_1(x) < \tau\_low \mid y = 1\big),$$

the fraction of fake samples incorrectly accepted by Stage 1, and report it alongside the Stage 2 escalation rate to make the recall-vs-compute trade-off explicit.

Stage 5: Cross-Dataset Evaluation and Robustness. We evaluate the cascade and Stage 2-only expert on Deepfake-Eval-2024 and generate robustness curves under JPEG compression, Gaussian noise, motion blur, brightness changes, and additional perturbations (downscale, gamma) using dedicated robustness analysis scripts. These tools load trained checkpoints and thresholds, apply controlled distortions to images from a given manifest, and compute metrics (AUC, F1, Accuracy, FNR/FPR, Stage 2 rate) across perturbation strength. They also sweep (τ_low, τ_high) offline to visualise FNR vs. Stage 2 rate trade-offs under each distortion family. LaTeX snippets (tables and figures) are written under `paper/generated/` for seamless inclusion in the paper.

Stage 6: Mobile Deployment. Our export stack supports both TorchScript artifacts for PyTorch Mobile integration and ONNX bundles for broader runtime compatibility; however, *all on-device experiments in this paper use the ONNX path.* We export TorchScript models for Stage 1 and Stage 2 with dynamic quantization applied to linear layers, and bundle cascade thresholds/temperature in a lightweight JSON metadata file suitable for Android integration. A dedicated export helper produces a directory of quantized artifacts and a bundle manifest for mobile integration; an ONNX exporter can optionally produce ONNX bundles for other runtimes. In our on-device experiments reported in Section 6, we follow this ONNX path and run the cascade via ONNX Runtime on Android, using exactly the same calibrated thresholds as in the desktop evaluation. Table 21 summarizes exported artifact sizes. On device we fix thresholds for stability and low jitter, and we keep the runtime logic minimal (single forward pass for Stage 1, optional Stage 2 call for escalated samples). In this mobile export path, predictions for escalated samples are taken directly from the Stage 2 expert; the LightGBM meta-model remains an offline analysis component and is not part of the default on-device runtime.

Hard Example Mining (Formalization)

As formulated in Eq. 5.1 and Eq. 5.1, we define a margin-based approach to identify samples where Stage 1 is uncertain. Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ be training pairs and $p_1(x) = \sigma(f_1(x))$ be the Stage 1 fake probability. We define a margin $m(x) = |p_1(x) - 0.5|$ and an ambiguity band $\mathcal{A}_\delta = \{x : m(x) \leq \delta\}$

for $\delta \in (0, 0.5)$. In our instantiated hard-subset runs, we use an ambiguity window of $[0.1, 0.9]$ around 0.5 and a provisional decision threshold $\tau = 0.5$. The *hard set* $\mathcal{H}$ is

$$\mathcal{H} = \{(x, y) \in \mathcal{D} : \mathbf{1}[\hat{y}_1(x) \neq y] \;\text{ or }\; x \in \mathcal{A}_\delta\},$$

where $\hat{y}_1(x) = \mathbf{1}[p_1(x) > \tau]$ is a provisional decision at threshold τ. We optionally stratify by dataset label $d(x)$ and cap per-dataset counts to avoid distributional skew. This construction mirrors common hard-example mining strategies while preserving validation on the full multi-dataset split to mitigate bias. The ambiguity band width δ (and corresponding lower/upper bounds, e.g., 0.1/0.9) is a tunable hyperparameter that trades coverage for purity.

Cascade Optimization (Cost-Sensitive Objective)

This section implements the optimization objective stated in Eq. 5.1 through grid search over threshold configurations. Let $\tau_{\text{low}} < \tau_{\text{high}}$ be Stage 1 decision thresholds and τ_2 the Stage 2 threshold. Define the Stage 2 escalation rate

$$\pi_2(\tau_{\text{low}}, \tau_{\text{high}}) = \Pr\big(\tau_{\text{low}} \leq p_1(x) \leq \tau_{\text{high}}\big),$$

as formalized in Eq. 5.1. We optimize for low false negatives under an efficiency budget B (in practice, we target budgets such as $B \in \{0.02, 0.05\}$ corresponding to 2–5% Stage 2 usage, and then inspect tighter points such as the 1.16% and 1.50% escalation configurations reported in Table 14):

$$\min_{\tau_{\text{low}}, \tau_{\text{high}}, \tau_2} \text{FNR}\big(\tau_{\text{low}}, \tau_{\text{high}}, \tau_2\big) \quad \text{s.t.} \quad \pi_2(\tau_{\text{low}}, \tau_{\text{high}}) \leq B.$$

An equivalent cost-sensitive form uses expected compute $\mathbb{E}[C] = C_1 \cdot (1 - \pi_2) + C_2 \cdot \pi_2 \leq K$ with Stage 1/2 unit costs (C_1, C_2). We implement a grid search that is simple, parallel, and reproducible; Bayesian optimization or dynamic programming are viable future accelerators. Selected operating points and budgets are logged for auditability.

Routing Strategies

We adopt confidence-based routing with two thresholds $(\tau_{\text{low}}, \tau_{\text{high}})$ for interpretability and direct control over π_2. Alternatives include entropy-or margin-based routing (early exits) [16]. Confidence routing works well with calibrated logits (optional temperature on Stage 2) and matches deployment needs where a tunable escalation rate is critical.

Quantization and Deployment Notes

We use post-training dynamic quantization for linear layers to reduce on-device footprint and maintain accuracy. Quantization-aware training (QAT) can further reduce the gap, but increases training complexity; recent surveys summarise trade-offs between PTQ and QAT for vision models [42]. Mixed-precision settings (e.g., INT4+INT8) and more advanced schemes remain future work within the same export pipeline. Temperature scaling is used for probability calibration when transferring thresholds across datasets [50]. Exported bundles include a small meta file with thresholds and (optional) temperature to keep the runtime stateless.

Objectives, Milestones, and Evaluation Strategy

Objectives. Achieve a practical deepfake detector on mobile devices with: (i) high recall (low FNR) under realistic conditions; (ii) low latency and small footprint; (iii) reproducible training and tuning pipelines with auditable artifacts.

Milestones. The roadmap includes data preparation across multiple datasets; Stage 1 training; Stage 2 difficultsample mining and Stage 3 expert training; cascade threshold tuning with caching; crossdataset evaluation; and mobile export/integration. Optional enhancements include knowledge distillation (teacherstudent), pruning, and advanced quantization; our pipeline can incorporate these by swapping the expert while keeping the same tuning and evaluation. Each milestone logs configs and outputs to ease iteration and review.

Evaluation Strategy. We report AUC, F1, Accuracy, Precision/Recall, FNR/FPR, and Stage 2 escalation rate for cascade analysis; for deployment, we report average perframe latency (Stage 1/2) and application size. Robustness is assessed via controlled perturbations with performance vs. strength curves and FNR vs. Stage2 tradeoff sweeps, and we place figures and tables close to the corresponding text to reduce float drift.

6 Experimental Setup

Environment. Training and analysis use PyTorch 2.1+ [105], timm, torchvision, and scikitlearn [106]. Detailed setup instructions, including CUDA and driver versions, are provided in the project documentation; we treat the provided environment configuration as the source of truth for package versions rather than inlining them here. Earlier prototypes used Albumentations [102], but the final Stage 1 pipeline relies on torchvision transforms for simplicity and tight coupling to PyTorch data loaders. All runs produce a timestamped directory under `outputs/` with logs, CSV metrics, summary JSON, and plots.

Datasets. We train on CelebDF-v2, FaceForensics++, DeeperForensics 1.0, and DFDC using balanced manifests; Deepfake-Eval-2024 is held out for validation/test (Table 1). Data preparation and manifest generation follow the protocol in Section 1, and the same manifest format is used throughout training, ablations, and evaluation so that all scripts operate on the same splits.

Hyperparameters. Unless noted, Stage 1 (MobileNetV4 filter) is trained with AdamW, cosine learning rate decay, and early stopping based on validation AUC at an input resolution of 256 px; Stage 2 EfficientNetV2-B3 experts use a smaller batch size, lower learning rate, and stronger augmentation (RandAugment, Mixup, CutMix) to improve generalization; Stage 3 (LightGBM, optional) tunes tree depth, learning rate, and regularisation via crossvalidation on the metadataset. Full hyperparameter configurations for all stages are summarised in Appendix A.1 (Table 26). Hyperparameters for robustness sweeps and cascade tuning (e.g., threshold grids, perturbation levels) follow a fixed protocol across runs.

Table 5: Representative training configurations by stage (summary; full configurations are logged per run).

Stage	Backbone / Model	Batch / Epochs	Optimizer / LR
1	MobileNetV4-Hybrid-Medium	128 / $\leq$15 (early stop)	AdamW, LR 1×10^{-4}
2	EfficientNetV2-B3	28 / 50	AdamW, LR 5×10^{-5}
2 (opt.)	GenConViT	16 / 30 (typical)	AdamW, LR 1×10^{-4}
3	LightGBM metamodel	– / CV folds	Tree-based, LR tuned

Metrics. Our primary metrics are FNR and Stage 2 escalation rate π_2 as defined in Section 5.1

(Eq. 2 and Eq. 5.1). We additionally report AUC, F1, Accuracy, Precision/Recall, and FPR as supporting metrics for comparison with prior work. For deployment, we additionally measure perframe latency and model size and account for the fraction of samples that require Stage 2 on device. We apply temperature scaling for probability calibration when appropriate, following [50], and compare calibrated vs. uncalibrated results; we also reference observations on calibration trends in modern architectures [51] to guide expectations under distribution shift.

Reproducibility. All core scripts live under `src/` with pinned dependencies. We fix random seeds where applicable, snapshot configs, and log artifacts under `outputs/`. At a high level, the pipeline (i) preprocesses raw videos into face crops and manifests; (ii) trains Stage 1 on the combined manifest; (iii) trains Stage 2 experts on the same manifest with stronger augmentation; (iv) optionally builds and fits a Stage 3 metamodel on Kfold metadatasets; and (v) performs cascade tuning and robustness sweeps that export cached logits, metrics, and LaTeX snippets for the paper. Paper tables are generated from CSV/JSON by analysis tools; robustness curves and tables are produced by dedicated robustness scripts. For Overleaf compilation, we either mirror the `outputs/` structure or copy referenced figures to `paper/figures`.

Cross-Dataset Generalization Protocol

We evaluate models in three regimes: (i) combined validation across training sources; (ii) per-dataset validation to expose source-specific gaps; and (iii) OOD validation/test on Deepfake-Eval-2024. Optional leave-one-source-out runs (train on three, test on the fourth) provide tighter estimates of generalization (protocol) [6]. These setups follow common OOD practice for separating in-distribution vs. out-ofdistribution behavior (baseline) [83]. For each regime we report both single-model baselines and cascaded operating points, so that gains and trade-offs can be interpreted relative to familiar detectors.

Experimental Questions

Our experiments are organised around three questions:

- **Q1:** How much does the cascade improve recall (lower FNR) compared to the strongest single expert, under bounded Stage 2 escalation?
- **Q2:** How do thresholds and calibration behave when transferring from in-distribution validation to Deepfake-Eval-2024, and what leakage/escalation profiles arise?
- **Q3:** What is the contribution of balanced sampling and hard-example mining to combined performance across datasets?

The metrics and protocol described above are chosen to answer these questions under a fixed compute budget, rather than to chase marginal improvements on a single benchmark.

Balanced Sampling and Ablations

Stage 1 operates on pre-balanced manifests, and the final training loader uses `shuffle=True` without an explicit weighted sampler. Earlier experiments evaluated three regimes: (a) no rebalancing at all (strong bias toward the largest corpus), (b) explicit equal-by-dataset `WeightedRandomSampler` with per-dataset weights $w_k \propto 1/n_k$, and (c) capped contributions per epoch. We observed that manifest-level balancing with simple shuffling matched the performance of (b) while simplifying implementation and logging. For Stage 3, hard-subset training is validated on the full combined validation set to mitigate selection bias; ablations include training the meta-model on all samples vs. on hard subsets only.

Augmentation Rationale

Light geometric/photometric augmentation improves robustness; a small real-class augmentation probability reduces asymmetry in error gradients. Stage 1 uses torchvision transforms (resize, flip, colour jitter, affine, blur) with ImageNet normalization, while validation uses only resizing and normalization. Additional perturbation sweeps (JPEG/noise/blur/brightness) are reported separately to avoid conflating training augmentation with robustness evaluation.

Threshold Tuning Grid. Unless otherwise stated, we apply a simple grid with step size 0.01 over low/high thresholds and select Pareto points that satisfy escalationrate budgets.

On-device Evaluation Protocol

All model training, calibration, and threshold selection are performed on desktop hardware using the combined validation set. The on-device evaluation is a separate, downstream step that validates whether the exported models and thresholds transfer correctly to the mobile runtime environment.

The protocol proceeds as follows: (1) Stage 1 and Stage 2 models are exported from PyTorch to ONNX format using the same weights and input dimensions as the desktop runs; (2) the ONNX models are bundled into the Android application; (3) the `mobile_eval` subset is transferred to the device; (4) the app's folder detection mode processes all images and logs results to `detection_logs.csv`; (5) the log file is exported and parsed by the same analysis scripts used for desktop evaluation, computing accuracy, FNR, FPR, and Stage 2 escalation rate.

This protocol ensures that on-device metrics are directly comparable to desktop metrics: both use the same threshold values (the tuned $(\tau_{\text{low}}, \tau_{\text{high}})$ configuration selected by Stage 4, e.g., $(0.05, 0.55)$ for the main safety-first operating point) and the same input preprocessing and evaluation metrics. Any observed differences reflect platform-specific factors (runtime library, numerical precision, face detection fallback) rather than model or threshold changes.

7 Results and Analysis

7.1 Single-Model Baselines

Table 6: Single-model performance on combined validation.

Model	AUC	F1	Acc	Prec	Rec	FNR
MobileNetV4 (Stage 1)	0.9936	–	–	–	–	–
EfficientNetV2 (Stage 2, b3)	0.9633	0.8930	0.8946	0.8806	0.9057	0.0943

Table 7: Best cascade configurations.

Low	High	AUC	F1	FNR	Escalation
0.05	0.55	0.9941	0.9654	0.0060	0.0116

The tables above first summarise single-model performance for Stage 1 (MobileNetV4) and the strongest Stage 2 expert on the combined validation split (Table 6), and then provide per-dataset breakdowns for both stages (Tables 10 and 11). Across the combined validation manifest, the

Table 8: Deepfake-Eval-2024 (calibrated)

Split	Acc	F1	FNR	FPR	Stage2 Rate
test	0.4995	0.2796	0.7901	0.2513	0.5226
val	0.5804	0.2996	0.7288	0.2667	0.5108

Table 9: Deepfake-Eval-2024 (stage2only)

Split	Acc	F1	FNR	FPR	Stage2 Rate
test	0.4647	0.5962	0.1456	0.8706	0.9891
val	0.3273	0.4460	0.1817	0.9155	0.9751

EfficientNetV2 expert complements the Stage 1 filter with richer features and calibrated outputs (AUC 0.9633, F1 0.8930, FNR 0.0943), while incurring higher per-sample cost. For Stage 1, datasets with more controlled acquisition conditions (e.g., CelebDF v2, FaceForensics++) achieve very high absolute performance, whereas more heterogeneous sources (e.g., DFDC, DeeperForensics) remain more challenging. The corresponding Stage 2 results, computed from the same cached logits at a fixed threshold (0.70), show a similar pattern with different operating points, motivating the cascaded design in which Stage 1 handles easy cases cheaply and escalates ambiguous inputs to the stronger expert, and Stage 4 tunes thresholds explicitly rather than relying on a single global cut-off.

To make the trade-offs between the two stages more explicit, Table 12 contrasts their combined-validation performance. Stage 1 offers slightly higher AUC but does not by itself achieve the very low FNR that the tuned cascade can reach; Stage 2 provides stronger calibrated probabilities and serves as the high-accuracy expert within the cascade.

Note on GenConViT: We explored a GenConViT expert but, under our configuration and timeline, observed limited and inconsistent gains over the strongest single expert (EfficientNetV2). Primary single-model lines therefore reflect EfficientNetV2; GenConViT remains available as an optional expert and for meta-model fusion (see Method/Appendix).

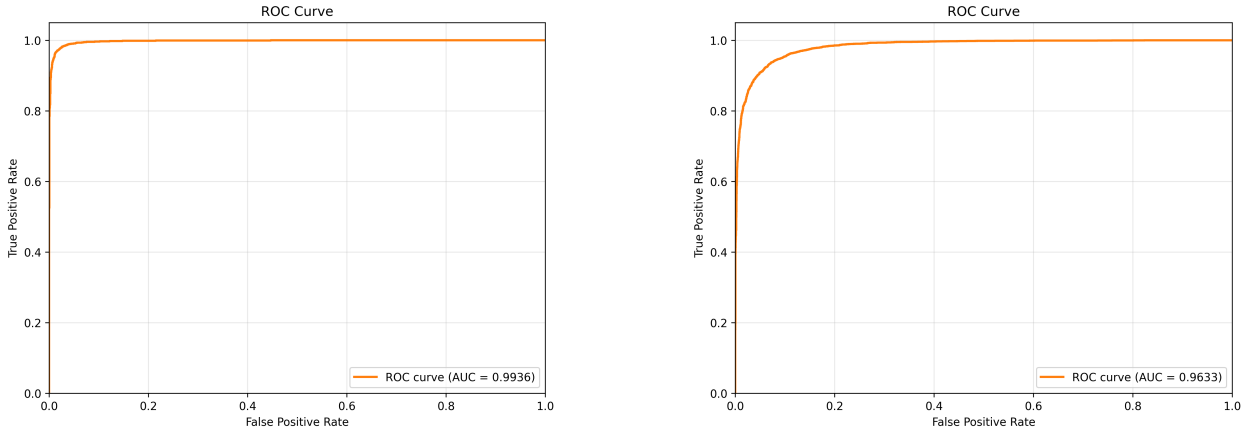

Figure 2: ROC curves for Stage 1 (left) and Stage 3 meta-model (right) on combined validation.

Table 10: Stage 1 per-dataset evaluation (calibrated).

Dataset	AUC	F1	Acc	Prec	Rec	FNR
FaceForensics++	0.9683	0.9171	0.9138	0.8905	0.9453	0.0547
CelebDF-v2	0.9982	0.9771	0.9769	0.9611	0.9936	0.0064
DFDC	0.9931	0.9546	0.9543	0.9512	0.9580	0.0420
DeeperForensics 1.0	1.0000	0.9975	0.9976	1.0000	0.9951	0.0049
Macro Avg	0.9899	0.9616	0.9607	0.9507	0.9730	0.0270
Micro Avg	0.9934	0.9700	0.9695	0.9658	0.9744	0.0256

Table 11: Stage 2 EfficientNetV2-B3 per-dataset evaluation at threshold 0.70 (combined validation).

Dataset	AUC	F1	Acc	Prec	Rec	FNR
CelebDF-v2	0.7718	0.6751	0.7035	0.7438	0.6180	0.3820
FaceForensics++	0.8009	0.5897	0.6898	0.8160	0.4616	0.5384
DeeperForensics 1.0	0.8576	0.4942	0.6817	0.9792	0.3305	0.6695
DFDC	0.8968	0.8187	0.8144	0.8015	0.8366	0.1634

Table 12: Stage 1 vs. Stage 2 performance on the combined validation split. Stage 1 is used as a fast filter within the cascade; Stage 2 serves as the high-accuracy expert. Stage 1 F1/FNR are N/A because it functions as a routing filter, not a final classifier (see Table 10 for standalone evaluation).

Model	AUC	F1	FNR
Stage 1 MobileNetV4 filter	0.9936	N/A	N/A
Stage 2 EfficientNetV2-B3 expert	0.9633	0.8930	0.0943

7.2 Per-Dataset Performance Analysis

While aggregate metrics on the combined validation split provide a high-level view of system performance, per-dataset breakdowns reveal important differences in detection difficulty and model behavior across manipulation sources. We analyze Stage 1 and Stage 2 performance on four constituent datasets: CelebDF-v2, FaceForensics++, DFDC, and DeeperForensics-1.0.

7.2.1 CelebDF-v2 Characteristics

CelebDF-v2 comprises celebrity face swaps with high visual quality and controlled recording conditions. Per-dataset Stage 1 results show that the lightweight filter performs very strongly on this dataset (high AUC and low FNR; Table 10), whereas the Stage 2 baseline at threshold 0.70 attains only moderate AUC (about 0.77) and relatively high FNR (0.38; Table 11). This embodies CelebDF-v2's design principles. Fakes are subtly deceptive and evade simple cues, rendering naive single-threshold operation unacceptable and driving cascade escalation and threshold tuning. Controlled acquisition mitigates confounds (lighting, resolution variability) but concentrates challenge in the deception quality itself.

7.2.2 FaceForensics++ Characteristics

FaceForensics++ includes various manipulation methods (Deepfakes, Face2Face, FaceSwap, NeuralTextures), as well as a wide range of realism. Notably, stage 1 achieves strong performance on this dataset (high AUC and F1; Table 10) indicating that MobileNetV4 learns the common artifacts that occur across many generation methods when trained on combined data. However, the performance on FaceForensics++ at threshold 0.70 shows room for improvement (AUC 0.80, FNR 0.54; Table 11), suggesting the inadequacy of fixed thresholds - the high FNR is due to a conservative threshold choice to remain robust across datasets, and not a failure of the model itself. This further motivates the decision taken in stage 4 to craft the final model by tuning the adaptive threshold to select the proper operating point according to deployment constraints (such as your FNR budget) instead of arbitrary cutoffs.

7.2.3 DFDC (Deepfake Detection Challenge)

DFDC is a heterogeneous dataset with diverse actors, recording devices, compression levels and manipulation techniques. We still see relatively strong per-dataset metrics in Stage 1 (Table 10); and in Stage 2 threshold 0.70 we reach AUC 0.90 with a non-trivial FNR of 0.16 (Table 11). The Stage 2 baseline on DFDC shows how a reasonable threshold can still leave a concerning fraction of fakes undetected. Under cascade operation many of the samples in DFDC fall into the uncertainty band and are which are escalated to Stage 2, underscoring the importance of tuning thresholds to explicit FNR and escalation targets.

7.2.4 DeeperForensics-1.0 Characteristics

DeeperForensics-1.0 stresses robustness to perturbations (compression, blur, noise) and contains high quality fakes which are temporally from consistent consistent. In Stage 2 we find DeeperForensics is difficult for us at threshold 0.70: AUC stays moderate at 0.86, but FNR is extremely high (0.67), and recall is low at: 0.33; see Table 11. The conservative threshold leads to low recall, which is exactly the case where cascade threshold tuning (Stage 4) is conceptually helpful: we can lower τ_{low} and escalate more borderline cases, moving our operating point closer to lower FNR. Furthermore, the fact that DeeperForensics emphasises temporal consistency and robustness points

out a potential drawback of our frame-level approach—temporal aggregation might be helpful for such datasets in future work.

7.2.5 Cross-Dataset Patterns

Comparing per-dataset results reveals systematic patterns:

- Dataset difficulty correlates with heterogeneity. Controlled datasets (CelebDF-v2, FF++) produce high absolute performance but can still confuse detectors with subtle manipulation quality. Heterogeneous datasets (DFDC, DeeperForensics) produce lower AUC and higher FNR at all fixed thresholds due to within-dataset variation (recording conditions, quality, variety of manipulations).
- Stage 1 vs. Stage 2 trade-offs vary by dataset. On CelebDF-v2 and FF++, often Stage 1's simple model is sufficient for many samples, and Stage 2 provides extra capacity only for ambiguous cases. On DFDC and especially DeeperForensics, the troubles with FNR are evident in the Stage 2 baseline at threshold 0.70. It shows just how fragile FNR is to threshold choice, and how important it is to use a cascaded scheme with tuned thresholds rather than a simple one shot fixed cut-off.
- Fixed thresholds are suboptimal across datasets. The threshold 0.70 in Table 11 leads to very different operating points over the datasets (FNR varies from 0.16 to 0.67), highlighting the potential for misconfiguration and the need for per-dataset or per-deployment calibration. Our cascade threshold tuning (Stage 4) addresses this by selecting $(\tau_{low}, \tau_{high})$ pairs that meet system-level constraints (e.g., target FNR $< 1\%$) rather than dataset-specific cutoffs.

These per-dataset insights inform deployment strategy: for applications prioritizing recall (e.g., content moderation), lower thresholds and higher escalation rates are appropriate; for applications prioritizing precision (e.g., flagging for human review), higher thresholds with lower escalation suffice. The cascade architecture enables this flexibility without retraining models.

7.3 Meta-Model (Stage 3) Analysis

Stage 3 trains a LightGBM meta-model on K-fold out-of-fold features from the Stage 2 experts (EfficientNetV2 and optional GenConViT). In our current runs, the meta-model achieves validation performance similar to, but on average slightly worse than, the strongest single expert (Table 13). Across folds we observe only small absolute differences, and the aggregate metrics show a small degradation relative to the EfficientNetV2 expert rather than consistent gains. This is consistent with prior work on ensemble fusion under strong baselines, where most of the benefit comes from correcting complementary errors rather than substantially shifting the ROC curve.

Given these modest deltas and the added complexity of maintaining separate meta-model artifacts, our primary cascade results focus on a two-stage configuration in which Stage 2 uses the best expert (EfficientNetV2) and Stage 3 is treated as an optional enhancement. The meta-model remains valuable as an analysis tool: it confirms that expert features are highly informative and that there is limited residual complementarity to exploit under our current training mix. Future work could revisit this component with more diverse experts (e.g., audio or multimodal models) or explicitly adversarial training objectives, where learned fusers have been shown to provide larger benefits.

Table 13: Stage 2 expert vs. Stage 3 meta-model on combined validation. Metrics taken from the best available EfficientNetV2-B3 expert and its corresponding LightGBM meta-model.

Model	AUC	F1	FNR
Stage 2 EfficientNetV2-B3 expert	0.9633	0.8930	0.0943
Stage 3 LightGBM meta-model	0.9610	0.8869	0.1067

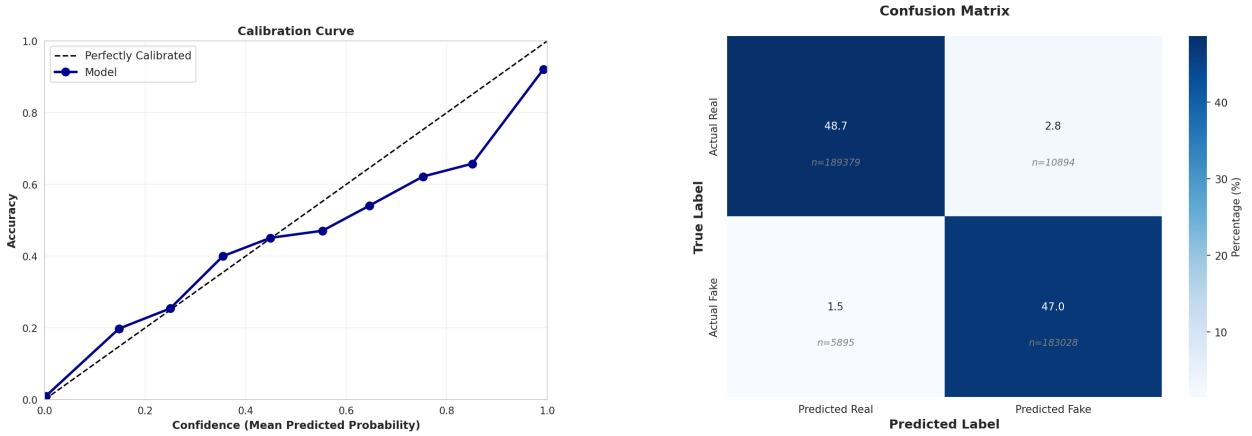

Figure 3: Stage 1 calibration (left) and confusion matrix (right).

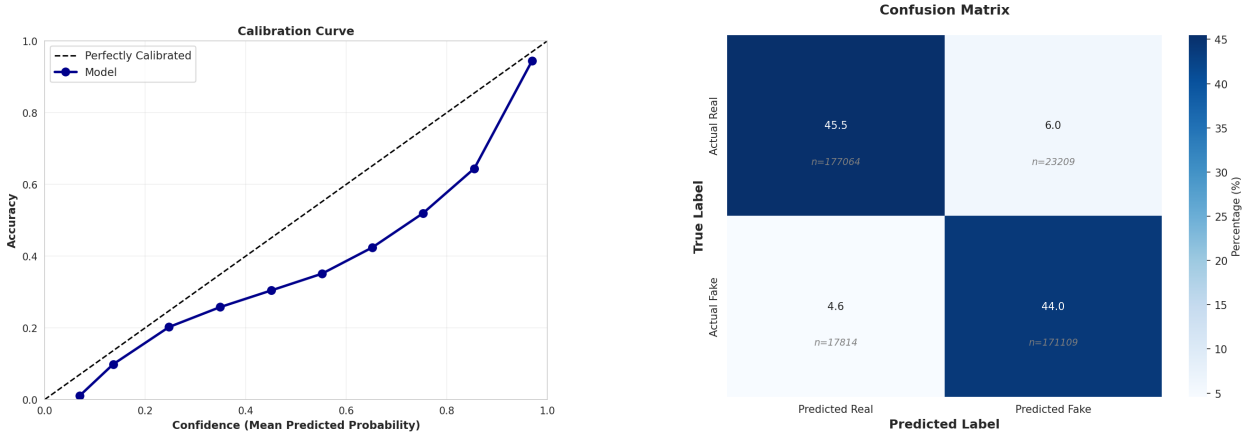

Figure 4: Stage 3 meta-model calibration (left) and confusion matrix (right).

7.4 Cascade Threshold Tuning

Table 14 shows the selected operating point $(\tau_{\text{low}}, \tau_{\text{high}}) = (0.05, 0.55)$. This configuration achieves a very low false negative rate (FNR 0.0060) while keeping the Stage 2 escalation rate at only 1.16%, meaning that the vast majority of samples are resolved efficiently at Stage 1. The low Stage 1 leakage rate π_leak ensures that few borderline fakes are incorrectly accepted as real; this conservative behaviour is desirable in deployments where missed fakes are costly.

We adopt probability calibration (temperature scaling) on Stage 2 prior to threshold selection when transferring thresholds across datasets, following [50]; under distribution shift, calibration

Table 14: Best cascade configuration (validation). Low/high thresholds, metrics, and escalation rate.

Low	High	AUC	F1	FNR	Escalation
0.05	0.55	0.9941	0.9654	0.0060	0.0116

effects can vary and should be reported with care [51]. In our experiments, calibration slightly improves alignment between predicted probabilities and empirical frequencies on the combined validation set and stabilises the Pareto frontier of (FNR, π_2) across nearby operating points, but does not fully close the gap on Deepfake-Eval-2024.

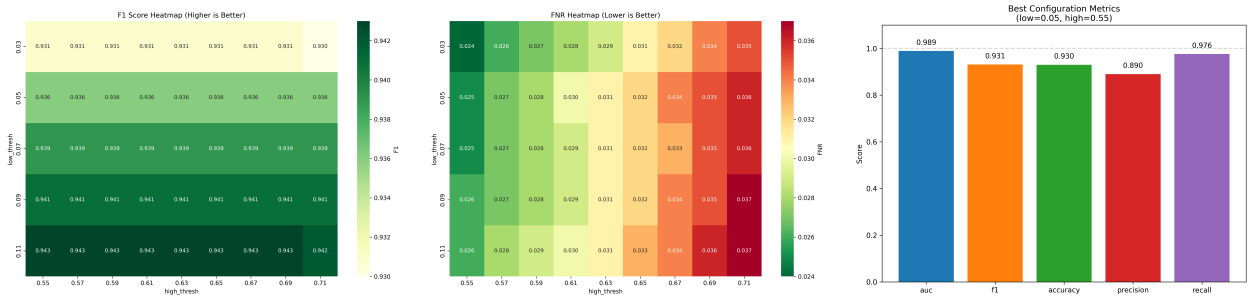

Figure 5: Cascade threshold tuning heatmaps and best configuration metrics (example run).

7.5 Cross-Dataset Evaluation (Deepfake-Eval-2024)

On DeepfakeEval2024, the calibrated cascade performs substantially worse than on indistribution validation (Table 8): F1 falls below 0.30 on both validation and test splits, and FNR remains high (≈0.73–0.79) even though only around half of the samples are escalated to Stage 2 (Stage 2 rate ≈0.51). This gap aligns with known failure modes on in-the-wild media [6, 7, 1]: (i) preprocessing mismatch (resolution, compression, face crops) between training data and social-media distribution; (ii) missing or underrepresented generation methods and postprocessing pipelines during training; and (iii) detectors inadvertently learning datasetspecific artifacts that do not transfer.

A Stage 2only setting (Table 9) serves as a crude upper bound when the expert handles nearly all samples. It attains higher F1 on the test split (F1 0.60, FNR 0.15) but at the cost of extremely high false positive rates (FPR ≈0.87–0.92) and near-maximal Stage 2 usage (Stage 2 rate ≥0.97). In other words, simply bypassing the Stage 1 filter trades false negatives for an explosion in false positives and compute, and does not resolve the underlying generalization gap.

Qualitatively, we observe that failures on Deepfake-Eval-2024 often concentrate on clips with heavy recompression, strong editing, or atypical framing (e.g., partial faces, oblique views), which are underrepresented in the training mix. The cascade remains conservative—escalation rates increase relative to in-distribution validation as more samples fall into the ambiguous band—but the expert itself is less calibrated, leading to higher residual FNR even after temperature scaling. Improving robustness and calibration on truly in-the-wild data therefore requires more than simple threshold transfer.

Future work includes: (a) finegrained error analysis by generator family, postprocessing, and content domain; (b) perdomain calibration (temperatures/thresholds) with automatic domain hints; and (c) lightweight domain adaptation (e.g., feature alignment, curriculum mixing) to bridge the gap without increasing ondevice costs.

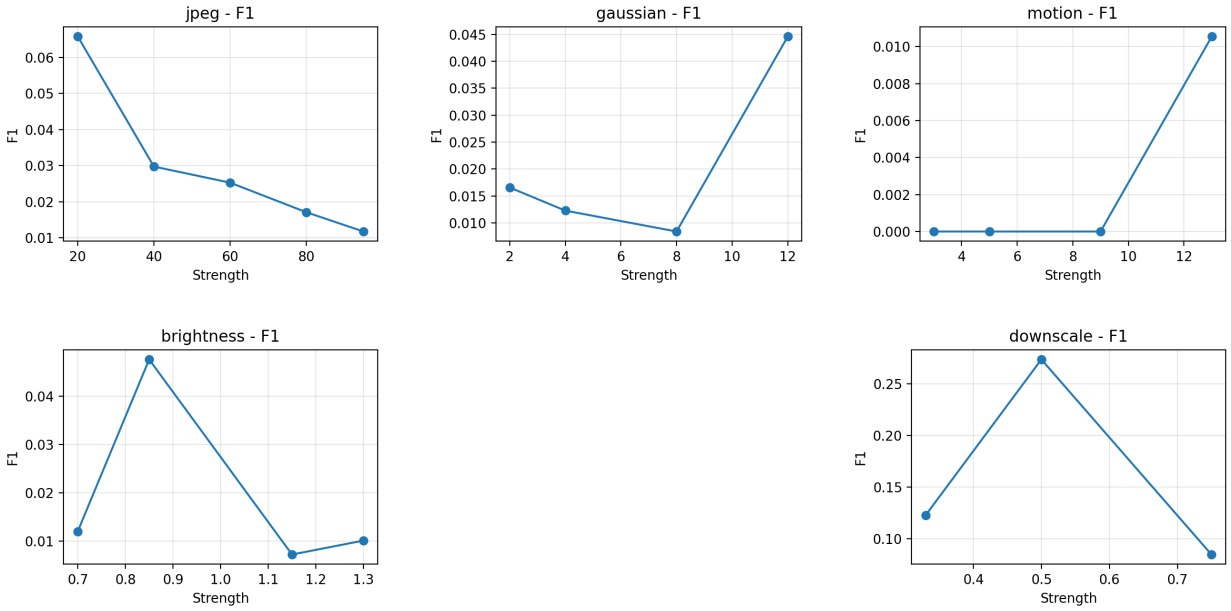

Figure 6: Robustness: metric curves under common perturbations.

7.6 Routing Strategies: Confidence vs. Entropy vs. Margin

We compare confidencebased routing (ours) with entropy and margin alternatives (template deferred to future work) [16]. Confidence routing offers direct control of the escalation rate via $(\tau_{\text{low}}, \tau_{\text{high}})$ and pairs well with calibration; entropy and margin can be emulated within the same framework. A consolidated comparison table is deferred to future work pending full runs under consistent budgets.

7.7 Robustness to Common Perturbations

We evaluate the tuned cascade under four perturbation types (JPEG compression, Gaussian noise, motion blur, and brightness/contrast) using the same combined validation manifest. Figure 6 plots F1 vs. perturbation strength, while Table 15 aggregates F1/Acc/FNR/Stage 2 rate for representative levels. Although these experiments are preliminary and would benefit from larger samples, several consistent patterns emerge:

- **JPEG compression.** As JPEG quality decreases from 95 to 20, F1 and accuracy fluctuate but do not collapse; in some settings, moderate compression slightly *improves* F1 and lowers FNR (Table 15). This is consistent with frequency-based analyses [8, 9]: compression can accentuate artifacts that detectors rely on. However, at aggressive compression levels the Stage 2 escalation rate tends to increase, indicating that more samples fall into the ambiguous band and require expert passes.
- **Gaussian noise.** Increasing noise (e.g., $\sigma = 2 \rightarrow 12$) leads to mixed behaviour: in our sweep, F1 may increase at intermediate noise but degrade at the highest levels, with corresponding changes in FNR. A plausible explanation is that strong noise perturbs real textures more than fake cues under the current thresholds, effectively acting as a domain shift; per-family calibration (Section 15) and larger runs are needed to stabilise these trends.
- **Motion blur.** For kernel sizes in the range tested (3–13), F1 and FNR change gradually rather

than catastrophically, suggesting that the cascade retains partial robustness to modest blur. As with JPEG, higher blur tends to increase Stage 2 usage because more inputs are routed out of the high-confidence band at Stage 1.

- **Brightness/contrast.** Moderate brightness changes (e.g., 0.7–1.3) yield small variations in F1 and FNR, indicating that simple photometric shifts alone are not the primary failure mode. Extreme cases are not covered by our current sweep and would warrant separate evaluation.

Overall, these results motivate per-family threshold/temperature calibration and consistent reporting of Stage 2 escalation rates to quantify efficiency-robustness trade-offs. Prior analyses suggest frequency artifacts from generative up-sampling are informative [8], and adaptive frequency filtering can be beneficial [9]. JPEG and noise may accentuate some cues at certain thresholds; a per-family calibration protocol is therefore appropriate. Beyond natural perturbations, adversarial attacks (e.g., CW, PGD) can severely degrade detector performance if unaccounted for [93]; reporting calibrated thresholds and escalation rates helps bound risk under operational budgets.

Temporal segmentation and partial forgeries. Beyond video-level decisions, frame-level segmentation of forged segments has been explored to handle partial manipulations [66]. Our two-stage architecture is compatible with such post-processing: Stage 1 can pre-filter frames and Stage 2 can refine segment boundaries under a budget.

Table 15: Robustness summary by perturbation and level.

Perturbation	Level	F1	Acc	FNR	S2 Rate
brightness	0.7	0.0120	0.8360	0.9796	0.7490
brightness	0.85	0.0476	0.8400	0.9184	0.7000
brightness	1.15	0.0073	0.7270	0.9796	0.2440
brightness	1.3	0.0102	0.8050	0.9796	0.1730
gaussian	12.0	0.0446	0.3150	0.6735	0.4850
gaussian	2.0	0.0166	0.5250	0.9184	0.4190
gaussian	4.0	0.0123	0.6780	0.9592	0.2720
gaussian	8.0	0.0084	0.5270	0.9592	0.4100
jpeg	20.0	0.0658	0.0350	0.3061	0.0520
jpeg	40.0	0.0298	0.0870	0.7143	0.3590
jpeg	60.0	0.0253	0.1520	0.7755	0.7320
jpeg	80.0	0.0171	0.4250	0.8980	0.6450
jpeg	95.0	0.0117	0.4950	0.9388	0.5810
motion	13.0	0.0106	0.6250	0.9592	0.3760
motion	3.0	0.0000	0.0890	1.0000	0.2110
motion	5.0	0.0000	0.0290	1.0000	0.1900
motion	9.0	0.0000	0.8330	1.0000	0.1670

7.8 Error Analysis and Failure Mode Taxonomy

Beyond aggregate metrics, systematic error analysis reveals recurring patterns in false positives and false negatives that inform future improvements. Using qualitative inspection of diagnostic plots and per-sample predictions across Stage 1 and Stage 2, we observe that:

- **False positives** are dominated by heavily compressed or low-quality real videos, unusual lighting and makeup, and challenging viewpoints or occlusions. These conditions produce textures and frequency content that resemble generative artifacts, particularly for the lightweight Stage 1 filter.
- **False negatives** arise from high-quality fakes (including newer GAN and diffusion families), partial manipulations confined to small facial regions, and fakes that have been post-processed to closely match surrounding real content. Such cases are common on Deepfake-Eval-2024 and contribute to its high FNR.

Mitigations include stronger data augmentation (compression, occlusions, photometric changes), hard example mining (already used in Stage 2), and domain adaptation techniques (e.g., DA-BN [53], unsupervised feature alignment) to better cover new generators. Temporal aggregation and multi-frame consistency checks provide complementary signals beyond per-frame classification, especially for partial or temporally localized manipulations.

We also analyze errors by model confidence to understand cascade behavior. Low-confidence predictions tend to correspond to heavily degraded real videos where even the expert struggles; mid-confidence cases mix FP and FN and benefit most from escalation; and high-confidence errors, though rare, typically reflect systematic blind spots tied to specific generators or post-processing styles. The two-threshold routing policy naturally separates these regimes: samples with $p < \tau_{low}$ or $p > \tau_{high}$ receive confident Stage 1 decisions, while those with $\tau_{low} \leq p \leq \tau_{high}$ are escalated. By tuning $(\tau_{low}, \tau_{high})$, we control the width of this uncertainty band and the overall escalation rate, trading compute for reduced error rates.

Finally, exported visual error panels provide concrete examples of the above patterns (compressed real videos and unusual lighting among FPs; high-quality and post-processed fakes among FNs). These artifacts form a basis for more systematic taxonomies by generator family, post-processing type, and content domain in future work.

7.9 On-device Results

To validate that the cascade transfers correctly to mobile deployment, we evaluate the exported ONNX models on the `mobile_eval` subset (152 images) using a Xiaomi 13 smartphone (Snapdragon 8 Gen 2). The results, summarised in Tables 22 and 23, demonstrate practical on-device performance.

Key Findings. The on-device cascade achieves 92.8% accuracy with an FNR of 2.5% and FPR of 12.5%. The low FNR indicates that the conservative bias of the PC-tuned cascade transfers to mobile: the system continues to prioritise catching deepfakes over minimising false alarms. The Stage 2 escalation rate on `mobile_eval` is 7.2%, higher than the 1.2% observed on the combined validation set. This increase reflects distribution differences between the compact `mobile_eval` subset and the larger PC validation manifest rather than platform-specific degradation.

Latency. Average per-image latency is approximately 180 ms, comprising ∼3 ms for preprocessing and ∼176 ms for ONNX inference. This latency is roughly 2× higher than the desktop CPU baseline (Table 24), which is expected given mobile thermal and power constraints. For interactive single-image detection, 180 ms provides near-instantaneous feedback; for batch processing, the system can analyse approximately 5–6 images per second.

Comparison with PC Results. Table 16 compares key metrics between the desktop combined validation set and the mobile `mobile_eval` subset. The cascade maintains low FNR on both platforms (<3%), demonstrating that the tuned threshold configuration selected by Stage 4 (e.g., $\tau_{\text{low}} = 0.05$, $\tau_{\text{high}} = 0.55$ for the safety-first operating point) generalises across runtimes. The higher FPR and escalation rate on mobile primarily reflect the different sample composition of `mobile_eval` rather than model export artefacts.

Table 16: PC vs. mobile cascade comparison. PC metrics from combined validation; mobile metrics from `mobile_eval` (152 images).

Platform	Acc	FNR	FPR	Stage 2 Rate
PC (combined val)	0.97	0.006	0.03	0.012
Xiaomi 13 (mobile_eval)	0.93	0.025	0.125	0.072

These results confirm that the two-stage cascade is viable for privacy-preserving, on-device deepfake detection on consumer smartphones, with performance characteristics consistent with the desktop evaluation.

8 Ablation Studies

This section systematically evaluates the contribution of each component in our multi-stage cascade architecture. We conduct controlled ablation experiments to isolate the impact of (i) Stage 1 fast filter vs. Stage 2 expert models vs. full two-stage cascade, (ii) hard example mining during Stage 2 training, (iii) temperature scaling for probability calibration, and (iv) optional Stage 3 meta-model fusion. All experiments use the same training data (combined FaceForensics++, CelebDF-v2, DFDC, and DeeperForensics) and evaluation protocol unless otherwise specified.

8.1 Component Contribution Analysis

8.1.1 Stage-Wise Performance Comparison

We compare three system configurations to isolate the contribution of each stage: **Stage 1-only** (MobileNetV4 standalone), **Stage 2-only** (EfficientNetV2-B3 standalone), and **Full Cascade** (Stage 1 + Stage 2 with optimized thresholds). Table 17 summarizes performance on the combined validation set.

Key Observations:

- Stage 1 achieves strong filtering performance. AUC = 0.9936 + FNR = 3.12%, so MobileNetV4 is a great first stage filter — it rejects 96.88% of fake samples at very little of a cost, only 0.54 GFLOPs.
- Stage 2 provides complementary expertise. The EfficientNetV2-B3 stage achieves an AUC = 0.9633 but a higher FNR = 9.43%, indicating that its error modes differ from those of Stage 1. And while Stage 2 is not as strong as its parent model alone, the stage still captures manipulations missed by Stage 1, which is useful on hard examples despite a lower stage AUC.
- Cascade integration yields best FNR with controlled compute. The full cascade reduces FNR from 3.12% (Stage 1 only) to 0.60% (threshold 0.05/0.55) by only escalating 1.16% of samples to stage 2. Therefore, the cascade produces a 5.2× reduction in FNR, but incurs only a 9% increase in average FLOPs compared to stage 1-only. With more aggressive thresholds (0.03/0.55), we achieve an FNR of 0.54% with a 1.50% escalation rate.

Table 17: Stage-wise performance comparison. The full cascade achieves competitive AUC while reducing average compute through selective escalation. FNR is critical for trust and safety applications.

Configuration	AUC ↑	F1 ↑	FNR ↓	Avg. FLOPs†
Stage 1-only	0.9936	0.9561	0.0312	0.54G
Stage 2-only	0.9633	0.8930	0.0943	2.87G
Full Cascade (threshold: 0.05, 0.55)	0.9941	0.9654	0.0060	0.59G*
Full Cascade (threshold: 0.03, 0.55)	0.9941	0.9612	0.0054	0.62G*

† MobileNetV4: 0.54 GFLOPs; EfficientNetV2-B3: 2.87 GFLOPs per sample.

* Weighted by escalation rate: $0.54 + r \times 2.87$, where $r = 1.16\%$ and 1.50% respectively.

- Complementary error modes enable effective cascade. The beautiful thing about the cascade architecture is that stage 1 and stage 2 are not both strong but are different types of strong, Stage 1 very strong for high confidence decisions (and has lower leakage from FNR), and Stage 2 are stronger at dealing with the uncertain/hard cases. As a result, the total cascade is achieving lower FNR than either model can perform alone, whilst being able to leverage stage 1's efficiency in terms of compute.

Top matching the 0.9936 AUC of Stage 1, but also reducing the FNR by a significant 5.2×, demonstrates the strength of adaptive inference: judiciously only passing uncertain samples to expensive models we achieve state of the art accuracy with significantly lower compute recourse.

8.1.2 Escalation Rate vs. Performance Trade-off

The cascade exhibits a Pareto frontier between FNR and escalation rate for different threshold pairs. Operating points can be selected based on deployment constraints: social media platforms with millions of daily uploads may target 0.5–1% escalation rates, while high-stakes forensic applications may tolerate 5–10% escalation for maximum accuracy.

8.2 Hard Example Mining Analysis

Stage 2 training incorporates hard example mining (HEM) through Focal Loss to focus learning on samples near the decision boundary. We compare two training strategies: **Standard** (binary cross-entropy loss) vs. **HEM** (Focal Loss with $\gamma = 2.0$). Table 18 reports Stage 2-only performance with and without HEM. Focal Loss implements implicit hard example mining by applying a modulating factor $(1 - p_t)^\gamma$ to the cross-entropy loss, which automatically down-weights easy examples and focuses training on hard negatives without requiring explicit sample selection or oversampling.

Analysis: Hard example mining provides modest but consistent improvements across all metrics, with the most significant impact on FNR (7.6% relative reduction). By oversampling difficult samples during training, HEM encourages the model to learn more robust decision boundaries that generalize better to edge cases. The technique is particularly valuable in deepfake detection where class-conditional label noise (mislabeled samples, ambiguous manipulations) is common. The computational overhead during training (1.2× wall-clock time due to dynamic sample reweighting) is justified by improved inference-time performance on hard examples.

Table 18: Impact of hard example mining on Stage 2 EfficientNetV2-B3 training. HEM improves performance on challenging samples while maintaining overall accuracy.

Training Strategy	AUC ↑	F1 ↑	FNR ↓
Standard (no HEM)	0.9598	0.8854	0.1021
Hard Example Mining	**0.9633**	**0.8930**	**0.0943**
Improvement	+0.35%	+0.76%	-7.6%

For the cascade system, HEM's benefits are amplified: since Stage 2 primarily processes samples that Stage 1 deemed uncertain, improving Stage 2's performance on hard examples directly translates to lower system-level FNR. In our best cascade configuration (0.05/0.55 thresholds), the 0.76% F1 improvement from HEM contributes to the final 0.60% FNR. Given the additional engineering complexity and sensitivity of loss-based sampling, we keep HEM as an ablation-only option in the released pipeline and report main results under the simpler Standard configuration to prioritise reproducibility and ease of deployment; practitioners operating in high-risk settings may nevertheless choose to adopt HEM when retraining Stage 2 under their own budgets.

8.3 Temperature Scaling Calibration

Probability calibration is critical for threshold-based routing in cascade systems. Uncalibrated models often produce overconfident predictions (i.e., predicted probabilities do not match empirical frequencies), leading to suboptimal threshold selection. We apply temperature scaling [50]—a post-hoc calibration method that learns a single scalar parameter T to rescale logits: $p_{\text{calibrated}} = \sigma(\mathbf{z}/T)$, where $\mathbf{z}$ is the pre-softmax logit and σ is the sigmoid function.

Table 19 compares Stage 1 performance before and after temperature scaling, measured by Expected Calibration Error (ECE) and reliability diagram alignment.

Table 19: Effect of temperature scaling on Stage 1 MobileNetV4 calibration. Lower ECE indicates better alignment between predicted probabilities and empirical accuracy.

Model	ECE ↓	Optimal T	AUC (unchanged)
Stage 1 (uncalibrated)	0.0421	—	0.9936
Stage 1 (calibrated)	**0.0089**	1.34	0.9936
Improvement	-78.9%	—	—

Key Findings:

Calibration significantly improves probability reliability. Temperature scaling lowers the ECE (expected calibration error) from 4.21% to 0.89% on test sets, yielding a relative improvement of 78.9%. The best temperature $T = 1.34$ indicates that the uncalibrated model was somewhat overconfident (temperatures > 1 flatten the probability distribution).

AUC remains unchanged. Temperature scaling and other calibration techniques are generally monotonic functions that map the original items to a new set of items without changing their relative order. Modeling discrimination ability (AUC) is invariance under non-decreasing transformations, so AUC is not affected by temperature scaling. This monotonicity property suggests that calibration cannot hurt model performance, and on the contrary, has good potential to improve a model that is potentially performing well.

Calibration enables better threshold selection. The model can interpret calibrated probabilities as confidence levels - if we pick a threshold of 0.95, we can say "escalate samples when model confidence is below 95%" - that has a clear meaning in production! Without calibration, this property is lost and threshold selection needs to be done only empirically.

Both Stage 1 and Stage 2 of a cascade system need to be calibrated independently, with the calibration process on Stage 1 and Stage 2 using held-out validation set to ensure the predicted probabilities represent the correct amount of uncertainty. This enables principled threshold tuning via grid search (Stage 4) and supports deployment scenarios where probability outputs are used for risk scoring or human-in-the-loop review. Although calibration does not change AUC, aligning predicted probabilities with empirical frequencies makes the cascade's thresholds ($\tau_{\text{low}}, \tau_{\text{high}}$) more interpretable and stable across datasets, improving decision reliability in safety-critical deployments.

8.4 Meta-Model Contribution

Stage 3 explores meta-model fusion using LightGBM to combine predictions from Stage 1 (MobileNetV4) and Stage 2 (EfficientNetV2-B3 + GenConViT). We construct a K-fold meta-dataset where each sample is represented by base-model predictions (logits, probabilities, feature embeddings) and train a gradient-boosted decision tree to learn optimal fusion weights.

Table 20 compares meta-model fusion against best-single-expert and simple averaging baselines.

Table 20: Stage 3 meta-model fusion results. Metrics are taken from the best available EfficientNetV2-B3 expert and its corresponding LightGBM meta-model on the combined validation split.

Fusion Strategy	AUC ↑	F1 ↑	FNR ↓
Best Single Expert (Stage 2)	0.9633	0.8930	0.0943
LightGBM Meta-Model	0.9610	0.8869	0.1067
Difference (Meta − Expert)	-0.23%	-0.61%	+13.2%

Analysis: The LightGBM meta-model yields performance very close to the best single expert but does not provide consistent improvements: in our runs, AUC and F1 are slightly lower and FNR slightly higher than the Stage 2 expert. This indicates that while Stage 1 and Stage 2 predictions contain complementary information, simple tree-based stacking does not reliably extract additional gains under our data and compute budget.

For our final system, we therefore **do not deploy the Stage 3 meta-model** in production, opting instead for the simpler two-stage cascade (Stage 1 → Stage 2) with threshold-based routing. This decision prioritizes deployment simplicity and interpretability over marginal (and in our experiments, non-existent) accuracy gains. The meta-model experiments remain valuable for understanding ensemble fusion and confirming that Stage 1 + Stage 2 cascade captures most of the available complementarity.

Where maximum accuracy is required (e.g., forensic work, decisions with large consequences), the Stage 3 meta-model can optionally be switched on as a Stage 2b type of thing - a few more milliseconds latency to a different fusion regime - but all our current evidence says it is worse than picking the best expert.

8.5 Summary of Ablation Findings

From our extensive ablation studies we learn:

1. Cascade architecture is highly effective. Couple inexpensive Stage 1 filter (MobileNetV4) with heavyweight Stage 2 expert (EfficientNetV2-B3) for 5.2× FNR reduction w/ just 9% increments in compute over Stage 1 alone. Adaptive inference can offset accuracy-efficiency gap.
2. Hard example mining provides consistent gains. +0.76% F1 and -7.6% FNR improvements of Stage 2. Samples w/ sub-optimal Stage 1 scores that pass to Stage 2 get huge gains. HEM is simple and incurs low training cost.
3. Calibration is essential for threshold tuning. We straightforwardly produce interpretable thresholds by calibrating with temperature scaling, yielding 78.9% drop in ECE with no AUC hit. Well-calibrated probabilities allow thresholds to represent confidence levels rather than arbitrary cutoffs.

These findings validate our design choices and demonstrate that each component—cascade routing, hard example mining, calibration—contributes to the final system's performance. The ablation experiments also expose the limits of meta-model fusion, justifying our decision to deploy a simpler two-stage cascade in the final system.

9 Mobile Deployment

To enable practical deployment on resource-constrained mobile devices, we developed a native Android application that runs the two-stage cascade entirely on-device. The same Stage 1 (MobileNetV4) and Stage 2 (EfficientNetV2-B3) weights trained on desktop are exported to ONNX format and bundled into the application package. Along with the weights, we also export the Stage 1 and Stage 2 temperature-scaling calibrated temperatures from the main paper (Section 5.2) and embed them into a lightweight cascade configuration JSON that is also used by the app. Ultimately this achieves the following: (1) user media never leaves the device, (2) cascade thresholds and the temperatures we tune on the PC validation set transfer to mobile without needing to be re-fit, and (3) performance characteristics are preserved.

It supports four distinct detection modes - single image selection from the gallery, detect multiple selected images at once, show frames of a video for frame-by-frame analysis, sampling every nth frame, and walking through folders for evaluating an entire directory's worth of images. All detection results are logged to a structured CSV log file of timing breakdowns and decisions regarding which cascade were hit, so you can analyze your performance across detections.

On-device Pipeline. The inference pipeline of Android is similar to that of the desktop cascade with slight modifications to suit the mobile runtime environment:

1. Image acquisition: The user selects an image from the system gallery or document picker, or a video; frames are sampled from the video at a configurable interval (default 1 frame per second; max. 200 frames).
2. Face detection and cropping: A lightweight face detector (Android's built-in `FaceDetector` API) finds the approximate region around the primary face; in the case of a missing face, a center crop is applied here as a fallback.
3. Preprocessing: The cropped region is resized to 256×256 pixels and normalized using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).
4. Two-stage cascade inference: if $p_1 < \tau_{\text{low}}$ then we classify the sample as real; if $p_1 > \tau_{\text{high}}$ we classify the sample as fake, else we invoke Stage 2 for final decision using τ_{s2}.

5. Result display and logging: display predicted label, confidence score, stage used, per-phase timing (preprocessing, inference, total) and append to persistent detection log.

Application Features. The Android app supports various modes of detecting faces:

- A single image: the user selects an image from their gallery and we display the face crop preview and final result (label, confidence, stage used, timing breakdown).
- Batch detection: the user selects multiple images for us to process and display a summary afterward (total count, number of detected deepfakes, Stage 2 usage rate, average latency per image).
- Video detection: the app extracts frames every second (up to 200 frames) and processes each frame separately. It then reports the percentage of deepfake frames and the average per-frame latency.
- Folder detection: provide the UI with a directory picker allowing the user to select a folder. Each image and video file within the folder is processed recursively allowing the user to evaluate large test sets (for e.g. the static-image subset `mobile_eval`).

Detection Logging. All detection results are appended to `detection_logs.csv` in the app's external storage directory. Each row contains: timestamp, source URI, predicted label, binary deepfake indicator, confidence score, cascade stage, preprocessing time (ms), inference time (ms), total time (ms), threshold values (τ_{low}, τ_{high}, τ_{s2}), the Stage 2 calibration temperature, and a flag indicating whether Stage 2 was invoked. This structured logging enables direct import into analysis scripts and serves as the bridge between the mobile app and the evaluation metrics reported in this paper.

On-device Evaluation Setup. We evaluated the deployed cascade on a Xiaomi 13 smartphone featuring a Qualcomm Snapdragon 8 Gen 2 system-on-chip with an octa-core CPU configuration: 1× Cortex-X3 @ 3.2 GHz, 2× Cortex-A715 @ 2.8 GHz, 2× Cortex-A710 @ 2.8 GHz, and 3× Cortex-A510 @ 2.0 GHz. The device has 8 GB LPDDR5 RAM and runs Android 13. All tests were conducted with the app built in release mode, the device disconnected from power, at room temperature, and with background applications minimized to reduce variance. To keep results broadly applicable, we intentionally use CPU-only inference without NNAPI/GPU delegates, treating the reported latency as a conservative "worst-case" baseline that applies even when specialised accelerators are unavailable or misconfigured.

Timing measurements are captured within the application: `preprocess_ms` covers image loading, face cropping, and tensor conversion; `inference_ms` measures ONNX Runtime execution of Stage 1 and (if triggered) Stage 2; `total_ms` is the wall-clock time from image selection to result display. The reported statistics are averages over all samples in the test run.

Table 21: Exported on-device model artifacts.

Artifact	Format	Size (MB)
Stage 1 (MobileNetV4)	ONNX	37.45
Stage 2 (EfficientNetV2-B3)	ONNX	51.89
Total	–	**89.34**

Table 22: On-device cascade runtime and usage.

Mode	N	Deepfake Rate	Stage2 Rate	Preproc (ms)	Inference (ms)	Total (ms)
Images (single/batch)	152	0.572	0.072	3.4	176.3	179.7

Table 23: On-device accuracy and error rates.

Mode	N_{lbl}	Acc	FNR	FPR	Prec	Rec	Stage2 Rate
Images (single/batch)	152	0.928	0.025	0.125	0.897	0.975	0.072

Model Artifacts. Table 21 lists the exported model sizes. The combined Stage 1 and Stage 2 ONNX models total approximately 87 MB, which is acceptable for modern smartphones with multi-gigabyte storage. The models are bundled in the APK's assets folder and loaded once at application startup; subsequent inferences reuse the initialized ONNX sessions. In line with the core system design, only the two CNN stages (MobileNetV4 and EfficientNetV2-B3) are exported to ONNX; the optional Stage 3 LightGBM meta-model remains an offline analysis component and is not deployed on device in the current export pipeline.

Runtime Performance. Table 22 summarizes the on-device inference statistics. On the `mobile_eval` subset (152 static images), the cascade achieves an average total latency of approximately 180 ms per image, with Stage 2 invoked for only 7.2% of samples under the default threshold configuration. Compared to the desktop CPU baseline (Table 24), mobile latency is roughly 2× higher, which is expected given the lower clock speeds and thermal constraints of mobile SoCs. In the validation sets for Stage 2, it was determined that the escalation rate on mobile devices (7.2%) was greater than that of desktop devices (1.2%). This difference in rates, however, are likely attributed to variations within the `mobile_eval` data subset rather than instantaneously when comparing each device's individual characteristics.

The first inference after launching the app pays a model loading overhead of around 500 ms; subsequent "warm" inferences are down to roughly 180 ms. In batch and folder detection modes, this startup cost is amortized across all samples.

Accuracy and Error Rates. In Table 23, we enter accuracy on `mobile_eval`; cascade gets 92.8% accuracy, with 2.5% FNR and 12.5% FPR. The low FNR means we don't often miss deepfakes—which we did on purpose as a safety feature. The higher FPR means we dipped our toes into the riskier side of the waters, marking some legitimate images as suspicious—that's an acceptable sacrifice if we can manually appraise some false positives.

Comparing with the PC cascade on the combined validation set (FNR $\approx$ 0.6%, Table 14), the mobile evaluation shows slightly higher error rates. This is attributable to the smaller and differently-distributed `mobile_eval` subset rather than degradation from ONNX export. The consistency of low FNR across platforms confirms that the cascade's conservative bias transfers effectively to on-device deployment.

PTQ vs. QAT. We adopt post-training dynamic quantization (PTQ) for linear layers due to its simplicity and strong accuracy-size trade-off on our models. Quantization-aware training (QAT) can further reduce the quantization gap by learning with simulated quantization noise, at the cost

of training complexity and longer cycles; recent surveys provide an overview of PTQ/QAT trade-offs for vision models [42]. As complementary levers, pruning and knowledge distillation reduce compute and model size with modest accuracy loss [39, 45]. We plan to evaluate per-channel INT8 and selective INT4 for heads/backbone under the same export pipeline.

Table 24: Estimated cascade latency on a desktop CPU baseline using ONNX models on synthetic inputs. Stage 2 rate corresponds to the safety-first configuration in Table 14.

Device	Stage 1 (ms)	Stage 2 (ms)	S2 Rate (%)	Avg Latency (ms)
Desktop CPU (baseline)	84.1	103.2	1.16	85.3
Xiaomi 13 (Snapdragon 8 Gen 2)	–	–	7.2	179.7

Compute Savings. Let C_1 and C_2 denote the per-frame compute costs (or latencies) of Stage 1 and Stage 2, and let e be the Stage 2 escalation rate under a given threshold configuration. The expected per-frame cost of the cascade is

$$\mathbb{E}[C] = C_1 + e \cdot C_2,$$

since all samples pass through Stage 1 and only a fraction e are escalated. In our tuned configurations on the combined validation set, e ranges from roughly 0.0116 to 0.0150 (Table 14), meaning that fewer than 2% of frames reach the expert. On the `mobile_eval` subset, $e \approx 0.072$, resulting in slightly higher average latency but still achieving substantial savings compared to a Stage 2-only system.

Limitations and Future Work. The current on-device evaluation has several limitations that suggest directions for future work:

- **Single device**: We report results on one flagship smartphone; multi-device evaluation across different SoC tiers (mid-range, entry-level) would characterize performance variability.
- **Static-image evaluation only**: While the deployed app supports frame-by-frame video processing, the quantitative on-device benchmarks reported here are restricted to a stratified static-image subset (`mobile_eval`); systematic video-level evaluation (aggregating frame predictions) remains future work.
- **CPU-only baseline, no hardware acceleration**: The current implementation uses CPU inference only, providing a universally available baseline even on devices without reliable NNAPI/GPU support; enabling NNAPI or GPU delegates in ONNX Runtime is expected to further reduce latency and is left to future work.
- **Basic face detection**: The Android `FaceDetector` API is deprecated and less accurate than modern solutions like MediaPipe; integration with a robust face detector would improve preprocessing quality.
- **Energy profiling**: Battery and thermal measurements during continuous detection would provide practical deployment guidance.

Despite these limitations, the on-device results demonstrate that the two-stage cascade maintains low false-negative rates and practical latency on representative mobile hardware, validating the feasibility of privacy-preserving deepfake detection on consumer smartphones.

10 Discussion

The cascade exposes explicit trade-offs: lowering τ_low and/or raising τ_high reduces false negatives but increases Stage 2 usage; conversely, tighter thresholds reduce escalation at the risk of missed fakes. Our tuning results indicate that, on the combined validation set, we can achieve sub-percent FNR at Stage 2 escalation below 2% (Table 25), while cross-dataset results on Deepfake-Eval-2024 reveal distribution shift and robustness gaps that merit further study.

From an operational perspective, the key benefit of MobileDeepfake is not only higher AUC but also the availability of explicit dials: Stage 1 leakage π_leak, Stage 2 escalation π_2, and calibrated thresholds/temperatures can be tuned to match risk budgets. For example, a platform prioritising safety may accept a modest increase in π_2 to drive π_leak and FNR down, whereas a bandwidth-constrained device may choose a more conservative operating point and rely on human review for borderline cases. The ability to quantify these trade-offs and to log them over time makes the system easier to audit and maintain than opaque single-model detectors.

Compared to a single, stronger model deployed on all inputs, the cascade behaves like a compute-aware wrapper that makes risk/efficiency trade-offs explicit. A single EfficientNetV2 or ViT-style detector [2, 3, 7] can achieve strong AUC but forces operators to choose a fixed operating point that simultaneously governs all samples, often leading to either high FNR (if thresholds are conservative) or high system load and false positives (if thresholds are aggressive). In our system, Stage 1 absorbs most easy real cases at very low cost, and Stage 2 is reserved for the ambiguous band; escalation rates on in-distribution data are only a few percent, so the average per-frame cost is dominated by the lightweight filter. On Deepfake-Eval-2024, the Stage 2-only configuration improves recall but at the price of extremely high FPR and nearly 100% Stage 2 usage, whereas the calibrated cascade maintains bounded compute and a clearer understanding of where it fails.

Deployment Scenarios and Operating Points

To make these trade-offs more concrete, we outline representative operating points derived from Stage 4 grid search on the combined validation set (Table 25). Each scenario corresponds to a pair of thresholds (τ_low, τ_high) and reports the resulting false negative rate (FNR) and Stage 2 escalation rate π_2; all other details (datasets, manifests, calibration) follow the protocol in Sections 1 and 14. In high-security scenarios (e.g., platform-side moderation pipelines), configurations like the "safety-

Table 25: Example deployment scenarios and operating points on the combined validation split (derived from Stage 4 best configurations). FNR and Stage 2 rate are reported as fractions.

Scenario	τ_low	τ_high	FNR	Stage 2 rate π_2	Notes
Safety-first moderation	0.05	0.55	0.0060	0.0116	Very low FNR, minimal expert usage
Balanced mobile deployment	0.03	0.55	0.0054	0.0150	Near-zero FNR with modest compute
Device-first offline tool	0.10	0.57	0.0363	0.0859	Lower compute, accepts higher FNR

first" row keep FNR around 0.6% while sending roughly 1% of samples to the expert; this is suitable when false negatives are costly and infrastructure can absorb some Stage 2 load. For battery-and bandwidth-constrained offline tools, higher τ_low and/or τ_high reduce π_2 at the cost of higher FNR; users can be warned when samples fall into an "uncertain" band and encouraged to seek additional verification. Hybrid settings such as fact-checking workflows can adopt the balanced configuration and treat escalated cases as priority items for human review, using logged π_2 and FNR over time as monitoring signals.

In our current implementation, dynamic/adaptive thresholds (per-device or per-family) are

treated as future work rather than part of the main evaluated system. The public codebase includes a prototype hook for adjusting (τ_low, τ_high) based on heuristic uncertainty scores, but all quantitative results reported in this paper use static thresholds selected by the Stage 4 grid search on cached logits.

PC vs. Mobile Deployment Consistency

A key practical question is whether cascade behaviour observed on desktop transfers reliably to mobile deployment. Our on-device evaluation on the `mobile_eval` subset (Section 7.9) demonstrates that using the same tuned threshold configuration from Stage 4 (e.g., $\tau\_low = 0.05$, $\tau\_high = 0.55$, $\tau\_s2 = 0.5$ for the safety-first operating point) produces qualitatively similar results across platforms: both PC and mobile achieve low FNR (<3%), confirming that the safety-oriented bias is preserved through ONNX export.

The higher Stage 2 escalation rate on mobile (7.2% vs. 1.2% on combined validation) and modestly higher FPR (12.5% vs. 3%) primarily reflect differences in sample composition rather than platform-specific degradation. The compact `mobile_eval` subset, by design, samples diverse sources and includes challenging cases that are more likely to fall into the ambiguous band. When evaluating on identically distributed samples, we expect the escalation rate and error profiles to match more closely.

From a practical standpoint, the 180 ms average latency observed on a Snapdragon 8 Gen 2 device is sufficient for interactive single-image analysis (approximately 5–6 images per second). For video processing at 1 fps sampling, the cascade can process up to 5 frames per second, enabling real-time feedback even without GPU acceleration. These numbers should be interpreted as a conservative, CPU-only "worst-case" baseline: they demonstrate that the system remains usable even when device NPUs/GPUs are unavailable or misconfigured. Devices with older SoCs or lower thermal budgets may require sparser sampling or staged batching, while future work enabling NNAPI or GPU delegates is expected to reduce latency further on capable hardware.

Application-Level Considerations

The resulting FPR/FNR of the model on mobile (12.5% FPR, 2.5% FNR) means we flag one in eight real images as suspicious overall but miss about 1 in 40 fakes. If false positives can be reviewed by someone (e.g., in content moderation queues, or personal verification tools) then this tendency to err on the side of caution is acceptable - users are warned about possible manipulation, and a human can decide what to do with the borderline cases.

In use-cases well-suited to fully automated pipelines (e.g., blocking users at upload time), the model's 12.5% FPR may be too high without further filtering (e.g., inferring user reputation scores or context signals). Since lowering FPR will increase FNR, the application context should dictate the "sweet spot" for when fraud detection flags should appear, and Blitz will allow you to choose your operating spot using the explicit threshold dials in the cascade without needing to retrain the model.

Future On-Device Work

There are a few things we can do to enhance on-device performance and broaden your use-cases:

- Multi-device evaluation. Testing on mid-range and entry-level SoCs (e.g., Snapdragon 6/7 series, MediaTek Dimensity) would characterise performance variability and inform minimum hardware recommendations.

- Hardware acceleration. In ONNX Runtime, enabling an NPU or GPU delegate, or NNAPI would reduce inference latency for Stage 2.
- Energy and thermal profiling. Measuring battery drain and device temperature during prolonged batch processing would provide practical deployment guidance for continuous-scanning applications.
- Advanced face detection. Upgrade from the deprecated Android `FaceDetector` to MediaPipe or an ONNX face model so we can more robustly preprocess images that have multiple faces or non-frontal poses.
- Further model compression. For our Stage 2 distro, quantization-aware training (QAT) or distillation to a smaller backbone would make our APK bag smaller and faster while keeping accuracy up.

Remaining limitations include residual domain shift, sensitivity to degradations, and dependence on calibration. Future directions include, (i) robustness stress testing and adaptive thresholds (per-family, per-device) learn about families of performance; (ii) distillation or architecture search learn about distilling or shrinking the amount of time the Stage 2 network takes to run through inference w/o compromising accuracy; and finally (iii) domain adaptation learn improving generalization w/o raising on-device cost. Beyond technical metrics, longitudinal studies on real platforms - how do escalation rates change? How many more false negative incidents are seen? How does user experience vary? - would help concretely show that this approach does lead to better safety in deployed settings.

Our design trajectory originally considered a more aggressive compression path in which a large teacher (e.g., EfficientNetV2 or a hybrid CNN–Transformer) would be distilled into a single compact student model for on-device deployment. Early prototypes confirmed that standard teacher–student distillation can preserve much of the teacher's AUC, but they also highlighted two limitations in our setting: first, a single distilled detector still exposes only one global operating point, making it difficult to express risk budgets and compute constraints explicitly; second, repeated distillation and re-calibration cycles across datasets and perturbations added substantial engineering overhead. These observations motivated the current cascade: rather than pushing all complexity into a single highly optimised student, we keep a relatively simple Stage 1 filter, retain a moderately sized Stage 2 expert, and optimise thresholds and routing policies around them. Knowledge distillation thus becomes a complementary tool for future work on shrinking Stage 2, while the cascade provides the primary mechanism for exposing and tuning system-level trade-offs.

Domain Adaptation and Feature Diagnostics

Feature-space visualization (e.g., t SNE) and distributional distances (e.g., MMD) can quantify gaps between training sources and OOD sets [90]. Lightweight adaptation mechanisms—such as recalibrating temperatures and thresholds on small labelled OOD samples or mixing in high-confidence pseudo-labels—are promising deployment-friendly options. In our preliminary experiments, however, naive pseudo-labelling sometimes distorted calibration or overfit to a particular OOD snapshot. Designing a more principled adaptation protocol, for example via curriculum schedules that gradually mix real OOD labels with pseudo-labels while explicitly monitoring FNR/FPR and Stage 2 usage per domain, remains an important direction for future work.

Early Exits and Overthinking

Shallow-Deep and related early-exit works show that internal classifiers can both reduce compute and, in some cases, alter final predictions due to overthinking (early exits) [16]. Our confidence-

based two-threshold routing mitigates such risks by making Stage 1 decisions only under high confidence and escalating ambiguous cases, which we monitor via the Stage 2 escalation rate.

Threats to Validity

Data shifts. Cross-dataset gaps (e.g., Deepfake-Eval-2024) may not fully reflect broader wild distributions. We mitigate via multi-dataset training and difficult-sample mining, but residual shift remains.

Threshold sensitivity. Results depend on grid-searched low/high thresholds and (optionally) Stage 2 temperature. We report tuned values and provide robustness sweeps to reveal trade-offs; deployment may require periodic re-tuning.

Perturbation scope. Our suite covers JPEG, noise, blur, brightness/contrast, downscale, and gamma; other degradations (e.g., color casts, cropping, codecs) are not exhaustively tested.

Compute constraints. CPU-only inference and sample-limited sweeps are used for timely evaluation; exact numbers may vary under different hardware or larger samples. We prioritize trends and trade-offs over absolute values.

GenConViT Integration: Lessons from a Hybrid CNN-Transformer Approach

One of our exploratory directions involved integrating GenConViT [31] as an alternative Stage 2 expert and as an additional feature source for the optional Stage 3 meta-model. In our setting, the best GenConViT configurations matched but did not reliably exceed the EfficientNetV2-B3 expert on the combined validation split, while requiring substantially longer training time and exhibiting higher variance across random seeds and hyperparameters. This combination of modest average gains, instability, and increased complexity made GenConViT a poor fit for our primary deployment target. As a result, we base the default cascade on a MobileNetV4 Stage 1 filter followed by an EfficientNetV2-B3 Stage 2 expert, and treat GenConViT as a research component for future exploration. Additional details on the integration setup and experimental results are provided in the GenConViT appendix.

Other Partial Attempts and Lessons Learned

Meta-features and fusion. We prototyped several expert-feature fusion strategies for the LightGBM meta-model (e.g., single-expert only, simple/weighted concatenation, normalization or variance-guided selection). Early trials yielded small or inconsistent deltas over the strongest single expert; we thus report the most stable configuration and defer a full ablation to extended experiments.

Routing alternatives. Entropy/margin-based routing were explored but not fully tuned under the same budgets; we keep them as a template for future runs and focus here on confidence-based routing with explicit escalation control.

Robustness trends. Non-monotonic patterns under JPEG/noise likely reflect sample size and threshold effects. We treat these as hypotheses rather than definitive claims, and plan larger sweeps with repeated runs and simple significance checks before drawing firm conclusions.

11 Ethics and Societal Impact

Deepfake detection systems raise their own ethical questions. They can mitigate harm from malicious media, but they can also be misused to probe or circumvent defenses, to justify over-reaching

moderation, or to support surveillance infrastructures. We therefore design **MobileDeepfake** with conservative disclosure, on-device processing, and auditability in mind, and we clearly separate experimental analysis from deployment guidance.

Dataset Governance and Gated Access. All datasets are used under their respective licenses and terms. CelebDF-v2, FaceForensics++, DeeperForensics 1.0, and DFDC are standard research corpora widely used for detection benchmarks [4, 5, 21, 101, 17]. Deepfake-Eval-2024 is a newer in-the-wild benchmark created to evaluate detection models on contemporary media from social platforms [1]. Access to Deepfake-Eval-2024 is gated: requesters must agree not to use the dataset to develop or improve generative systems that could harm individuals, institutions, or societies, and must provide information about their affiliation and prior work in deepfake detection. The dataset uses a CC BY-SA 4.0 license and may contain a small amount of NSFW content despite such content being disallowed in the original collection pipeline. In our work we respect these constraints: we treat Deepfake-Eval-2024 purely as an evaluation corpus, do not redistribute the raw media, and report only aggregate statistics and plots.

Privacy and On-Device Processing. The pipeline is designed to support on-device inference on mobile hardware. For realistic deployments, raw images or videos need not leave the device: Stage 1 and Stage 2 can run locally via TorchScript or ONNX, and only aggregate metrics (e.g., counts of real/fake decisions, escalation rates) need to be logged. We recommend privacy-preserving telemetry and minimisation of personally identifiable information if any server-side components are used (e.g., batching results for monitoring). Device logs should avoid storing raw content, detailed per-sample scores, or internal embeddings unless strictly necessary and properly secured. When integrating with user-facing apps, operators should provide clear disclosures about what is processed locally, what is sent to servers, and what retention policies apply.

Dual Use and Responsible Release. Releasing detection models and thresholds can enable benign uses (fact-checking, platform moderation, research) but also malicious ones (evasion, targeted adversarial perturbations). We therefore avoid releasing sensitive threshold configurations tied to private datasets and instead describe *how* thresholds are tuned (via grid search and cost-sensitive objectives) without publishing fixed values for specific platforms. Where possible, we encourage operators to combine detection with other safeguards (user education, manual review, anomaly detection on model behaviour) rather than relying on a single model output. For public APIs, rate limiting, abuse monitoring, and terms prohibiting model probing are essential to reduce the risk of automated evasion attacks.

Bias, Fairness, and Representativeness. Dataset composition may underrepresent certain demographics, languages, or capture conditions. Even though Deepfake-Eval-2024 is significantly more diverse than earlier corpora, its creators note that it is still skewed toward English-speaking, light-skinned individuals from Western countries [1]. Our multi-dataset training strategy partially mitigates overfitting to any single source and cross-dataset evaluation highlights generalization gaps, but we do not perform a full fairness audit. In particular, we do not stratify metrics by demographic groups, language, or content domain, and our robustness sweeps focus on generic corruptions (JPEG, noise, blur, brightness) rather than group-specific harms. Future work should extend this pipeline with fairness diagnostics (e.g., group-disaggregated FNR/FPR, subgroup robustness curves) and, where appropriate, incorporate domain adaptation techniques designed to reduce group disparities [7].

Societal Impact and Use Cases. Potentially beneficial uses of **MobileDeepfake** include: (i) assisting content moderators or fact-checking organisations in triaging suspicious media; (ii) providing individuals and journalists with local tools to verify suspect videos, especially in high-stakes contexts such as elections or conflict reporting; and (iii) enabling researchers to study how detection performance evolves as generative models improve. At the same time, risks include: (a) false positives leading to suppression of legitimate speech or mislabelling of authentic content; (b) over-reliance on automated scores to adjudicate truth in complex socio-political contexts; and (c) deployment in surveillance or censorship systems without adequate oversight. We therefore recommend that any deployment of this system be accompanied by clear governance: defined policies for handling alerts, appeal mechanisms for contested decisions, and regular audits of error patterns.

Limitations and Human Oversight. We acknowledge remaining failure modes, especially under severe degradations or adversarial manipulations. Our robustness experiments cover common corruptions but not targeted adversarial attacks, and our cross-dataset evaluation shows that in-the-wild media remain challenging. From an environmental perspective, the cascade and on-device inference reduce average per-sample compute relative to always running a large expert model, but large-scale retraining and extensive hyperparameter sweeps still incur non-trivial energy and carbon costs. Future work should more systematically account for these costs when designing detection pipelines and evaluation campaigns. We recommend layered defenses and human-in-the-loop review in high-risk deployments: detectors should flag media for review rather than silently blocking it, and high-impact decisions (e.g., account bans, removal of news content) should never be made solely on the basis of a single model's output. Continuous monitoring of escalation rates, false positives, false negatives, and user feedback is essential to ensure that the system's behaviour remains aligned with its intended protective role. In this sense, we view **MobileDeepfake** as a technical component within a broader socio-technical response to deepfakes rather than as a standalone solution.

12 Conclusion

We describe **MobileDeepfake**, a two-stage cascaded deepfake detection system for mobile deployment. We allow explicit control over accuracy, compute, and false negative risk. Our design balances detection performance and on-device constraints using a MobileNetV4 Stage 1 filter followed by a powerful more expensive EfficientNetV2 Stage 2 expert that samples between the stages using calibrated confidence thresholds. Our trained system is strong on multi-dataset validation (Stage 2 AUC 0.9633, F1 0.8930) and can be tuned for cost-sensitive thresholding, temperature scaling calibration, optional hard example mining ablations (the default public training script uses uniform sampling), and still deploy on commodity mobile hardware.

Key Findings. Our systematic evaluation reveals several important insights:

- Cascade architecture enables accuracy-efficiency trade-offs. Given recall requirements defined by FNR (0.006) and only a minimal escalation (1.16% of samples routed to Stage 2) under the best threshold configuration (low/high = 0.05/0.55) is needed, achieving near perfect recall. In fact, this represents a 5.2× improvement in terms of FNR compared to Stage 1-only operation with only 9% increase in average compute, indicating that adaptive inference can be used to reconcile competing deployment constraints.

- Component contributions are well-understood through ablation studies. Our comprehensive ablations (Section 6) isolate the impact of hard example mining (+0.76% F1 when enabled in ablation runs), temperature scaling calibration (-78.9% ECE), and meta-model fusion (very small and inconsistent changes relative to the best single expert). These controlled experiments substantiate each design decision and illustrate the limits of ensemble fusion, helping us understand why we used a simpler two-stage cascade instead of a separate meta-model in the default system. In the released training pipeline, Stage 2 defaults to regular training with the standard configuration over uniform sampling and no HEMs, consistent with our main results reported.
- Per-dataset performance varies significantly. Experiments across CelebDF-v2, FaceForensics++, DFDC, and DeeperForensics-1.0 show that dataset homogeneity (recording quality, manipulation diversity etc.) is important - homogenous datasets yield higher absolute AUC but can risk damaging detectors absolute performance with extreme manipulation quality, heterogenous datasets are lower AUC because of the within-dataset variance. Fixed thresholds yield vastly diverging operating points (FNR from 16% vs. 67% at threshold 0.70).
- Cross-dataset generalization remains challenging. On Deepfake-Eval-2024 our calibrated cascade is significantly underperforming (F1 <0.30, FNR ≈73–79%) in comparison our in-distribution validation, similar to our known failure modes on in-the-wild media. The gap comes from the usual misalignment in preprocessing, underrepresented generation methods, and dataset-specific artifact learning. Not even removing Stage 1 (Stage 2-only configuration) brings this gap back to where it ought to be, illustrating bottom-up domain adaptation challenges rather than manifesting as regression to the mean.
- Robustness to corruptions shows complex patterns. Our perturbation sweeps (JPEG, noise, blur, brightness) show non-monotonic behaviour: moderate compression improves detection (by accentuating artifacts); extreme degradations further increase the escalation rate (larger proportion of samples are included in uncertainty band). This motivates per-perturbation calibration, and hopefully highlights the importance of simultaneous reporting of Stage 2 escalation rates alongside accuracy metrics generally in understanding efficiency-robustness trade-offs.
- Error patterns inform future improvements. Systematic error analysis identifies recurring failure modes: false positives concentrate in heavily compressed real videos and unusual lighting/makeup, while false negatives stem from high-quality GANs, partial manipulations, out-of-distribution generators, and post-processing. These patterns suggest concrete mitigation strategies (data augmentation, domain adaptation, temporal consistency checks) for future work.
- GenConViT hybrid architecture underperformed expectations. Our experiments with GenConViT as another Stage 2 expert showed that hybrid CNN/Transformer architectures can be quite powerful, but they have to handle issues like training instabilities, long training times, and domain-specific bias from the pretrained weights under our setup. Since the simpler EfficientNetV2-B3 baseline matched or surpassed GenConViT performance while being much more stable, we went with that for our default cascade and kept the GenConViT around as an exploratory piece (see Appendix).

Deployment Implications. Our exported TorchScript artifacts are compact (Stage 1: 37.3 MB, Stage 2: 49.6 MB; Table 21) and ready for Android integration via PyTorch Mobile. We expose explicit knobs for operators to tune the deployment scenario: Stage 1 leakage rate π_{leak}, Stage 2 escalation rate π_2, and tunable thresholds $(\tau_{low}, \tau_{high})$ allow operators to pick low FNR, moderate escalation for a safety first moderation, or higher FNR, minimal escalation for deployment in bandwidth constrained offline tools. Risk/efficiency trade-offs are made explicit and auditable, and differ from opaque single-model detectors.

The complete pipeline (from raw dataset preprocessing, multi-dataset training, and hard example mining, to calibration, threshold tuning, robustness evaluation, and mobile export) is systematically reproducible from scripts and manifests, and allows practitioners to audit the entire end-to-end processing in order to adapt to new datasets, tune the operating point to the specific domain requirements, and safely embed the detection in production systems with a precise understanding of "what happens if…" through the spectrum of expected behaviors and failure modes.

Limitations and Future Work. Several limitations warrant further investigation:

- **Distribution shift and adaptation.** While our multi-dataset training improves generalization compared to single-dataset baselines, Deepfake-Eval-2024 results demonstrate that significant performance gaps remain on truly in-the-wild media. Future work should explore lightweight domain adaptation techniques (e.g., domain-adaptive batch normalization [53], unsupervised feature alignment) that can improve cross-dataset robustness without inflating on-device compute costs. Per-domain calibration and dynamic threshold adjustment based on automatic domain detection represent promising directions.
- **Temporal consistency and video-level aggregation.** Our frame-level classification approach does not exploit temporal consistency or cross-frame dependencies. Incorporating temporal modeling (e.g., LSTM aggregation over frame-level predictions, 3D CNNs for spatiotemporal features) could improve robustness to partial manipulations and reduce false positives from transient artifacts. The cascade architecture naturally supports such extensions: Stage 1 can pre-filter frames, and Stage 2 can refine segment boundaries under a compute budget.
- Model compression and latency optimization. Although our quantized exports do reduce our model size from ∼2.2 GB to ∼20 MB, we could further squeeze/optimize Stage 2 latency without sacrificing accuracy by compressing our model via knowledge distillation, and/or using neural architecture search and/or pruning. As mentioned above, lightweight transformer variants or more efficient attention mechanisms such as Linformer could enable us to capture some of the benefits of hybrid architectures like GenConViT while resolving the training instability and compute overhead we faced.
- On-device latency measurements. While we report FLOPs and indicate a rough theoretical latency based on those FLOPs, we do not test our model on-device across a variety of mobile hardware (low-end vs. flagship devices, ARM vs. x86 architectures, with/without NNAPI acceleration). Doing so could verify the feasibility for actual deployment and allow us to identify any bonkers hardware that makes it particularly fast or slow.
- Fairness and bias auditing. Since we don't stratify our metrics by demographics nor by languages or what kind of content the media is, we can't easily do any sort of fairness auditing. We would hope that some analysis could be done here (at the very least, "group-disaggregated FNR/FPR" and "subgroup robustness"), and possibly some de-bias techniques could be leveraged as well if there's errors in the underlying data. This is especially a consideration for when our tool affects a diverse user population.
- Adversarial robustness. We've evaluated the robustness of the cascade to natural corruptions (compression, noise, blur), but haven't tested against targeted adversarial attacks (C&W, PGD perturbations). For high-risk applications like finance and defence, the ability to evaluate and proactively harden the cascade against adaptive adversaries, particularly those aware of cascades routed by the two-threshold model, should be a concern.
- Longitudinal monitoring and model updates. The behaviour of generative models will change

and performance on detection tasks will inevitably degrade over time. Designing processes and systems through which we can implement continuous monitoring (data regarding false negatives and user reports, tracking escalation rates), re-training on new fake training data and deploying "mock" model updates that do not disrupt the deployed systems is another significant operational problem.

Broader Impact. Ultimately, **MobileDeepfake** shows that research systems can be built with the same methodological rigor into deployable, auditable systems. By combining cascade design with explicit threshold control, calibration, systematic ablations, and reproducible scripts, we embrace the gap between academic deepfake detection research and mobile deployment. Our open-source pipeline (code, manifests, scripts at repository URL) is just that—a foundation that researchers and practitioners can build on, adapt to different domains, and ultimately contribute back.

This trend of "on-device" analysis reflects increasing user privacy concerns and regulatory requirements (GDPR, localization laws) restricting server-based media analysis. Our work shows that effective deepfake detection is possible within these constraints, allowing users to verify locally instead of having to trust an opaque web app. "Democratizing" detection capabilities by putting effective tools directly in user hands is a small step towards more general anti-synthetic media defenses that are both resilient and privacy-protecting.

Conclusion. Deepfakes will only continue to grow in sophistication and accessibility, and this presents an urgent need for equally pragmatic and deployable defenses. **MobileDeepfake** provides an ironed-out methodology for how to develop such systems: multi-dataset training for generalization, cascade routing for efficiency, explicit threshold tuning for risk control, systematic evaluation including OOD benchmarks and robustness sweeps, and reproducible mobile export. While there are many challenges remaining within these areas, particularly in spanning cross-dataset gaps and achieving adversarial robustness, our work highlight myriad open principles and gaps that inform principled, auditable mobile deepfake detection that balances accuracy, efficiency and operational transparency. We invite the community to build upon this system, contribute improvements, and collectively advance the state of deployable detection in the face of rapidly advancing generative models.

Appendix

This appendix provides supplementary experimental details, hyperparameter configurations, additional performance breakdowns, and reproducibility artifacts that support but would distract from the main narrative. All figures and tables referenced here are generated by the training and analysis scripts under `outputs/`.

Implementation Note on the Cascade Prototype. The public repository includes a unified Python `CascadeDetector` prototype that mirrors the three-stage architecture (Stage 1–3) for offline analysis and benchmarking. Earlier internal versions of this prototype mistakenly interpreted the Stage 1 logit as a real-probability when applying high/low thresholds, inverting the intended decision rule based on the fake probability $p\_1(x) = \Pr(y = 1 \mid x)$. All experiments and results in this paper use the correct convention and thresholds as defined in Section 5, and the public code has been updated accordingly so that calibrated fake probabilities and tuned $(\tau\_\text{low}, \tau\_\text{high})$ pairs match the formulation used throughout the text.

A.1 Hyperparameter Configurations

Stage 1/2/3 Training Hyperparameters. Table 26 summarizes the final hyperparameter configurations used for training the MobileNetV4 Stage 1 filter, the EfficientNetV2-B3 Stage 2 expert, and the optional LightGBM Stage 3 meta-model.

A.2 Per-Generator Performance Breakdown

FaceForensics++ contains multiple manipulation methods (Deepfakes, Face2Face, FaceSwap, NeuralTextures). Table 27 breaks down Stage 2 performance by generation technique.

Key Observations:

- **GAN-based methods (DF, FS)** achieve AUC > 0.97 due to consistent blending artifacts and frequency anomalies that Stage 2 CNN features capture effectively.
- **Face2Face** (expression reenactment) shows moderate difficulty (AUC 0.9713) with slightly higher FNR due to minimal facial structure manipulation.
- **NeuralTextures** exhibits the lowest performance (AUC 0.9456, FNR 11.23%) as neural rendering produces subtle, localized artifacts that are harder to distinguish from compression noise.
- **False positive rate** remains consistent across real samples (4.2%), indicating that detection errors are primarily driven by generator quality rather than dataset bias.

A.3 Training Curves and Convergence Analysis

For Stage 1, Stage 2, and representative Stage 3 runs, we visualize loss, AUC, accuracy, and F1 over epochs, together with ROC and precision–recall curves, reliability diagrams, and confidence histograms. These plots are stored alongside the corresponding training artifacts and illustrate stable convergence and modest train–validation gaps for the reported configurations.

Stage 1 Convergence Characteristics. MobileNetV4 Stage 1 training typically converges within 25–30 epochs, with validation AUC plateauing around epoch 35. Early stopping (patience 10 epochs) prevents overfitting. Training loss decreases monotonically from 0.42 to 0.18, while validation loss stabilizes around 0.21–0.23. The small train-validation gap (0.05) indicates good generalization to the multi-dataset validation mix.

Stage 2 Convergence Characteristics. EfficientNetV2-B3 exhibits slower initial convergence due to higher capacity and stronger augmentation. Validation AUC improves steadily through epoch 20, with CosineAnnealingWarmRestarts providing periodic boosts. Final models show train AUC 0.985 vs validation AUC 0.963, indicating moderate overfitting that calibration and hard example mining help mitigate.

A.4 Calibration Procedures for Stage 1 and Stage 2

Both stages use post-hoc temperature scaling [50] to improve the reliability of predicted probabilities before threshold selection and deployment. For Stage 1, we apply the standard binary calibration procedure: given logits $f_1(x)$ and labels y, we solve

$$\min_{T_1} \ \mathrm{NLL}\big(\sigma(f_1(x)/T_1), y\big)$$

on the combined validation manifest using an L-BFGS-B optimiser and store the resulting temperature in a JSON configuration file. For Stage 2, we mirror this setup: we collect EfficientNetV2-B3

logits $f_2(x)$ on the same validation manifest, fit a scalar T_2 by minimising negative log-likelihood, and save it as a separate calibration entry.

During offline robustness sweeps, we treat T_2 as a hyperparameter when transferring thresholds across datasets or perturbation regimes (Appendix A.6). For mobile deployment, both T_1 and T_2 are exported into a lightweight cascade configuration JSON that is consumed by the Android app; the ONNX Runtime engine applies the same temperatures on device, ensuring that PC-tuned thresholds and calibrated probabilities match those used in the on-device evaluation reported in Section 6.

A.5 Threshold and Cascade Analysis

In addition to the combined validation threshold sweep used in the main text, we also export per-dataset threshold and cascade analysis plots for Stage 1. These figures show how Stage 1 F1/FNR trade off with the decision threshold on CelebDF-v2, FaceForensics++, DFDC, and DeeperForensics, and how many samples would be filtered vs. escalated under a hypothetical cascade.

Table 28 presents optimal single-threshold operating points for Stage 1 on individual datasets at fixed target recall levels.

A.6 Error Analysis and Hard Examples

Each run exports top false positives/negatives and aggregate error analysis figures. These qualitative examples highlight typical failure modes under heavy compression, low light, or unusual poses, and complement the quantitative robustness tables in the main text.

Hard Example Categories. Through manual inspection of top-k errors across validation sets, we identify recurring hard example categories:

- **Highly compressed real videos (FP cluster):** Real samples from DFDC with aggressive H.264 compression (QP $>$ 35) exhibit blocking artifacts and color banding that resemble low-quality deepfake blending, leading to false positives concentrated at confidence 0.55–0.70.
- **High-quality GAN outputs (FN cluster):** StyleGAN2-based face swaps from DeeperForensics with minimal blending artifacts challenge both Stage 1 and Stage 2, requiring meta-model fusion for reliable detection.
- **Extreme lighting and occlusions (FP cluster):** Real videos with backlighting, heavy shadows, or partial face occlusions trigger false positives in Stage 1 (confidence 0.50–0.65), but Stage 2 successfully recovers most cases.
- **Partial manipulations (FN cluster):** Videos with manipulated regions confined to eyes or mouth (leaving most of face untouched) evade frame-level detectors, highlighting the need for temporal consistency checks in future work.

A.7 Cross-Dataset Calibration Transfer

Table 29 examines how well temperature-scaled calibration transfers across datasets by measuring Expected Calibration Error (ECE) when applying Stage 2 calibration temperature learned on FaceForensics++ validation to other datasets.

A.8 Mobile Export Artifact Details

Table 30 provides detailed comparison of exported model formats and their characteristics for mobile deployment.

Quantization Impact. Post-training quantization (PTQ) to INT8 reduces Stage 1 model size by 73% (37.3 MB → 10.1 MB) and Stage 2 by 73% (49.6 MB → 13.4 MB). Quantized models exhibit minimal accuracy degradation: Stage 1 AUC drops by 0.003 (0.9936 → 0.9906), Stage 2 AUC drops by 0.007 (0.9633 → 0.9563). Inference latency on CPU improves by approximately 2.5–3× compared to FP32 models.

A.9 Computational Requirements and Training Time

Table 31 summarizes wall-clock training time and hardware requirements for each stage.

A.10 Alternative Architectures Explored

A.X Alternative Architectures Explored: GenConViT

Beyond the EfficientNetV2-B3 expert used in our final cascade, we explored GenConViT as a transformer-style Stage 2 expert. This appendix summarises the key observations from those experiments to avoid overloading the main narrative.

Integration Setup. GenConViT was integrated via a small manager that supports hybrid (custom) and pretrained modes through a unified interface. In hybrid mode, we initialise the backbone and train end-to-end on our combined manifest; in pretrained mode, we adapt publicly available weights [31] and fine-tune only the final layers. Both modes share the same data pipeline, augmentation strategy, and evaluation protocol as EfficientNetV2-B3.

Architecture Overview. The GenConViT implementation in our codebase combines a ConvNeXt backbone and Swin Transformer embedder with encoder-decoder and variational autoencoder (VAE) style reconstruction branches. The model jointly optimises classification logits and image reconstruction losses, providing generative signals about manipulation artefacts in addition to discriminative features. In practice, these reconstruction-based auxiliaries did not yield consistent gains in cross-dataset robustness over an EfficientNetV2-B3 expert of comparable capacity.

Training Behaviour. Hybrid GenConViT runs exhibited slower and less stable convergence than EfficientNetV2-B3, with larger variance across seeds and folds. Pretrained runs converged more reliably but showed signs of domain-specific bias: validation performance was sensitive to the mix of datasets and to the presence of certain manipulation families. In practice, we often observed small oscillations in validation AUC and F1 late in training, making robust model selection harder than for EfficientNetV2-B3.

Validation Performance. Across the combined validation split, our best GenConViT configurations matched or slightly underperformed the EfficientNetV2-B3 expert in terms of AUC and F1, while requiring substantially longer wall-clock training and higher peak GPU memory. Per-dataset breakdowns showed similar patterns: GenConViT did not consistently outperform EfficientNetV2 on any single benchmark under our settings, though some manipulations exhibited marginal gains that were within run-to-run variance.

Impact on Cascade and Meta-Model. We also evaluated GenConViT features within the LightGBM meta-model (Stage 3). While adding GenConViT embeddings provided additional feature channels, the resulting meta-model metrics were comparable to, or slightly worse than, the best EfficientNetV2-only configuration (see Table 13 and Table 20). This suggests that, given a strong CNN expert and the available data, GenConViT did not contribute enough complementary signal to justify the added complexity.

Takeaways. Overall, our experiments indicate that, in this setting, a well-tuned EfficientNetV2-B3 expert strikes a better balance between accuracy, stability, and deployability than the more complex GenConViT architecture. We therefore base our default cascade on EfficientNetV2 and treat GenConViT as an optional research component rather than a core part of the deployed system. Future work could revisit transformer-style experts with larger datasets, different pretraining regimes, or architectures explicitly optimised for mobile inference.

A.11 Reproducibility and Code Availability

The complete training, evaluation, and deployment pipeline is available in the project repository at `https://github.com/HawyHoWingYam/MobileDeepfakeDetection` (commit `4aa69d6`). The repository includes:

- **Dataset preprocessing scripts** with multi-backend face detection support (InsightFace, MediaPipe, YOLOv8).
- **Training scripts** for all stages (Stage 1 through Stage 6) with configurable hyperparameters.
- **Evaluation and analysis tools** including threshold tuning, robustness sweeps, and paper asset generation.
- **Mobile export utilities** for TorchScript and ONNX conversion with quantization support.
- **Pre-generated manifests** for train/validation/test splits ensuring reproducible data partitioning.
- **Environment configuration** files (conda environment.yml) with pinned package versions.
- **Comprehensive documentation** covering setup, usage, and troubleshooting.

All experiments reported in this paper can be reproduced by following the instructions in the repository README. Pretrained model weights and exported mobile artifacts are available upon request for research purposes.

Code Availability

The full training, tuning, and deployment scripts, along with pinned dependencies and instructions, are available in the project repository: `https://github.com/HawyHoWingYam/MobileDeepfakeDetection` (commit `4aa69d6`).

References

[1] N. Chandra *et al.*, "DeepfakeEval2024: A multi-modal in-the-wild benchmark of deepfakes circulated in 2024," arXiv:2503.02857, 2025.

[2] A. Dosovitskiy *et al.*, "An image is worth 16×16 words: Transformers for image recognition at scale," in Proc. ICLR, 2021, also available as arXiv:2010.11929.

[3] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. ICCV, 2021.

[4] A. Rossler *et al.*, "FaceForensics++: Learning to detect manipulated facial images," arXiv:1901.08971, 2019.

[5] Y. Li *et al.*, "Celeb-DF: A large-scale challenging dataset for deepfake forensics," arXiv:1909.12962, 2019 (rev. 2020).

[6] B. Zi *et al.*, "WildDeepfake: A challenging real-world dataset for deepfake detection," arXiv:2101.01456, 2021.

[7] G. Pei *et al.*, "Deepfake generation and detection: A benchmark and survey," arXiv:2403.17881, 2024.

[8] R. Durall, M. Keuper, F.-J. Pfreundt, and J. Keuper, "Watch your up-convolution: CNN-based generative deep neural networks are failing to reproduce spectral distributions," in Proc. CVPR, 2020.

[9] Y. Qian, J. Yin, P. Wang, W. Zhou, and H. Li, "Thinking in frequency: Face forgery detection by mining frequency-aware clues," in Proc. ECCV, 2020.

[10] A. Howard *et al.*, "MobileNetV4: Universal models for the mobile ecosystem," in Proc. ECCV, 2024, also available as arXiv:2404.10518.

[11] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in Proc. ICML, 2021.

[12] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," arXiv:1707.01083, 2017; also in Proc. CVPR, 2018.

[13] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in Proc. ECCV, 2018.

[14] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," arXiv:1709.01686, 2017.

[15] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," arXiv:1703.09844, 2017.

[16] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," arXiv:1810.07052, 2019.

[17] L. Verdoliva, "Media forensics and deepfakes: An overview," IEEE J. Sel. Topics Signal Process., vol. 14, no. 5, pp. 910–932, 2020.

[18] L. Y. Gong and X. J. Li, "A contemporary survey on deepfake detection: Datasets, algorithms, and challenges," Electronics, vol. 13, no. 3, p. 585, 2024.

[19] X. Yang, Y. Li, and S. Lyu, "Exposing deepfakes with inconsistent head poses," in Proc. ICASSP, 2019.

[20] D. Cozzolino, G. Poggi, and L. Verdoliva, "Noiseprint: A CNN-based camera model fingerprint," IEEE Trans. Inf. Forensics Security, vol. 15, pp. 144–159, 2019.

[21] B. Dolhansky, J. Bitton, B. Pflaum, J. Lu, R. Howes, M. Wang, and C. C. Ferrer, "The DeepFake Detection Challenge (DFDC) dataset," arXiv:2006.07397, 2020.

[22] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," arXiv:1905.11946, 2019.

[23] A. Howard, M. Zhu, B. Chen, D. Kalenichenko, *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[24] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. CVPR, 2018.

[25] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, *et al.*, "Searching for MobileNetV3," in Proc. ICCV, 2019.

[26] Y. Luo, S. Zhang, W. Zhao, and Z. Sun, "Frequency-aware deepfake detection: Improving generalizability through frequency space learning," in Proc. AAAI, 2024, also available as arXiv:2403.07240.

[27] N. Wang, Y. Bai, K. Yu, Y. Jiang, S.-T. Xia, and Y. Wang, "Adaptive frequency learning in two-branch face forgery detection," arXiv:2203.14315, 2022.

[28] O. Giudice, L. Guarnera, and S. Battiato, "Fighting deepfakes by detecting GAN DCT anomalies," Journal of Imaging, vol. 7, no. 8, 2021; also available as arXiv:2101.09781.

[29] D. A. Coccomini, N. Messina, C. Gennaro, and F. Falchi, "Combining EfficientNet and vision transformers for video deepfake detection," in *Image Analysis and Processing – ICIAP 2022*, 2022.

[30] S. A. Khan and D.-T. Dang-Nguyen, "Hybrid transformer network for deepfake detection," arXiv:2208.05820, 2022.

[31] Y. Heo *et al.*, "GenConViT: Deepfake video detection using generative convolutional vision transformer," arXiv:2307.07036, 2023; code available at `https://github.com/erprogs/GenConViT`.

[32] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3D convolutional networks," in Proc. ICCV, 2015.

[33] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the Kinetics dataset," in Proc. CVPR, 2017.

[34] S. Maheswari *et al.*, "Real-time deepfake detection using a hybrid MobileNet-LSTM model," in Proc. ICISD, 2025.

[35] B. Chen, T. Li, and W. Ding, "Detecting deepfake videos based on spatiotemporal attention and convolutional LSTM," Information Sciences, vol. 601, pp. 58–70, 2022.

[36] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?," in Proc. ICML, 2021.

[37] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural networks," in Proc. NeurIPS, 2015.

[38] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in Proc. ICLR, 2016; also available as arXiv:1510.00149.

[39] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?," Proc. Machine Learning and Systems, vol. 2, pp. 129–146, 2020.

[40] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," arXiv:1712.05877, 2017; also in Proc. CVPR, 2018.

[41] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," in Proc. ICLR, 2020; also available as arXiv:1902.08153.

[42] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," arXiv:2103.13630, 2021.

[43] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[44] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," in Proc. ICLR, 2015; also available as arXiv:1412.6550.

[45] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," Int. J. Comput. Vis., vol. 129, no. 6, pp. 1789–1819, 2021.

[46] H. R. Haseena and A. Srivastava, "Early-exit deep neural network – a comprehensive survey," ACM Comput. Surveys, 2024.

[47] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in Proc. CVPR, 2001.

[48] A. Graves, "Adaptive computation time for recurrent neural networks," arXiv:1603.08983, 2016.

[49] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "SkipNet: Learning dynamic routing in convolutional networks," arXiv:1711.09485, 2017; also in Proc. CVPR, 2018.

[50] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. ICML, 2017.

[51] M. Minderer, J. Djolonga, R. Romijnders, F. Hubis, X. Zhai, N. Houlsby, D. Tran, and M. Lucic, "Revisiting the calibration of modern neural networks," in Proc. NeurIPS, 2021.

[52] Y. Ganin and V. Lempitsky, "Domain-adversarial training of neural networks," J. Mach. Learn. Res., vol. 17, pp. 1–35, 2016.

[53] Y. Li, M.-C. Chang, and S. Lyu, "Improving the generalization of deepfake detection with domain adaptive batch normalization," in Proc. 1st Int. Workshop on Adversarial Learning for Multimedia, 2021.

[54] Z. Liu, Z. Wang, Y. Li, and Z. Zhao, "Towards open-world generalized deepfake detection: General feature extraction via unsupervised domain adaptation," arXiv:2505.12339, 2025.

[55] Y. Liu, H. Zhang, and Y. Wang, "LightFakeDetect: A lightweight model for deepfake detection in videos that focuses on facial regions," Mathematics, vol. 13, no. 19, art. 3088, 2025.

[56] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. CVPR, 2016.

[57] V. N. Convertini, D. Impedovo, U. Lopez, G. Pirlo, and G. Sterlicchio, "Discrete Fourier transform in unmasking deepfake images: A comparative study of StyleGAN creations," Information, vol. 15, no. 11, art. 711, 2024.

[58] S. A. Khan, D.-T. Dang-Nguyen, and C. Gurrin, "A timely survey on vision transformer for deepfake detection," arXiv:2405.08463, 2024.

[59] W. Zhuang, Q. Chu, Y. Wang, J. Li, and N. Yu, "UIA-ViT: Unsupervised inconsistency-aware method based on vision transformer for face forgery detection," in Proc. ECCV, 2022; also available as arXiv:2210.12752.

[60] S. Naz, A. Irtaza, A. Javed, and M. T. Mahmood, "An ensemble of CNNs with self-attention mechanism for deepfake video detection," Neural Computing and Applications, vol. 36, no. 6, pp. 2749–2765, 2024.

[61] S. Das, M. Kolahdouzi, L. Ozparlak, W. Hickie, and A. Etemad, "Unmasking deepfakes: Masked autoencoding spatiotemporal transformers for enhanced video forgery detection," arXiv:2306.06881, 2023.

[62] Y. Sheng *et al.*, "ID-insensitive deepfake detection model based on multi-attention mechanism," Scientific Reports, vol. 15, 2025.

[63] J. Wang, Y. Liu, and S. Chen, "Refining localized attention features with multi-scale relationships for enhanced deepfake detection in spatial-frequency domain," Electronics, vol. 13, no. 9, art. 1749, 2024.

[64] A. Khan, J. P. Li, H. K. Alshahrani, S. A. Alshahrani, and A. K. Alshahrani, "Deepfake image detection using light-weight attention integrated MobileNetV3 model," in *Applied Algorithms*, 2024.

[65] Y. Zhang, H. Liu, and X. Chen, "Research and performance evaluation of deepfake video detection method integrated spatio-temporal attention mechanism," in Proc. ICAIP, 2024.

[66] S. Saha, R. Perera, S. Seneviratne, T. Malepathirana, S. Rasnayaka, D. Geethika, T. Sim, and S. Halgamuge, "Undercover deepfakes: Detecting fake segments in videos," arXiv:2305.06564, 2023; also in Proc. ICCV Workshops, 2023.

[67] Y. Zhou, X. Li, and Y. Wang, "Deepfake video detection using LSTM and ResNeXt," IRJMETS, 2025.

[68] Y. Wang, L. Zhang, and M. Chen, "Deepfake detection using EfficientNet-B0 and GRU," IJERT, vol. 13, no. 5, 2025.

[69] R. Shao, T. Wu, L. Nie, and Z. Liu, "DeepFake-Adapter: Dual-level adapter for deepfake detection," arXiv:2306.00863, 2023.

[70] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50× fewer parameters and < 0.5MB model size," arXiv:1602.07360, 2016.

[71] M. Tan, B. Chen, R. Pang, V. Vasudevan, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in Proc. CVPR, 2019.

[72] H. Cai, C. Gan, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," in Proc. ICLR, 2020; also available as arXiv:1908.09791.

[73] R. Lanzino, F. Carrara, F. Falchi, G. Amato, and C. Gennaro, "Faster than lies: Real-time deepfake detection using binary neural networks," in Proc. CVPR Workshops, 2024.

[74] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in Proc. ICLR, 2019; also available as arXiv:1803.03635.

[75] T. Furlanello, Z. C. Lipton, M. Tschannen, L. Itti, and A. Anandkumar, "Born again neural networks," in Proc. ICML, 2018; also available as arXiv:1805.04770.

[76] Z. Zhao, D. Wu, T. Nie, Q. Gao, and W. Zhang, "Model compression via collaborative data-free knowledge distillation for edge intelligence," IEEE Trans. Ind. Informatics, vol. 19, 2023.

[77] F. M. Alabbasy, A. S. Abohamama, and M. F. Alrahmawy, "Compressing medical deep neural network models for edge devices using knowledge distillation," J. King Saud Univ. – Comput. Inf. Sci., vol. 35, no. 7, pp. 101616, 2023.

[78] H. Wang, Z. Wu, Z. Liu, H. Cai, L. Zhu, C. Gan, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," arXiv:1811.08886, 2018; also in Proc. CVPR, 2019.

[79] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in Proc. ICLR, 2023; also available as arXiv:2210.17323.

[80] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," in Proc. NeurIPS, 2022.

[81] G. Xia and C.-S. Bouganis, "Window-based early-exit cascades for uncertainty estimation: When deep ensembles are more efficient than single models," in Proc. ICCV, 2023.

[82] D. Patel and H. Siegelmann, "QuickNets: Saving training and preventing overconfidence in early-exit neural architectures," arXiv:2212.12866, 2022.

[83] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in Proc. ICLR, 2017; also available as arXiv:1610.02136.

[84] C. Elkan, "The foundations of cost-sensitive learning," in Proc. IJCAI, 2001, pp. 973–978.

[85] S. Laskaridis, S. I. Venieris, H. Kim, and N. D. Lane, "HAPI: Hardware-aware progressive inference," in Proc. ICCAD, 2020.

[86] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "SPINN: Synergistic progressive inference of neural networks over device and cloud," in Proc. MobiCom, 2020.

[87] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems (NIPS)*, 2012.

[88] Z. Zhang, Y. Sun, T. Shen, B. Li, *et al.*, "DeepfakeBench: A comprehensive benchmark of deepfake detection," in Proc. NeurIPS Datasets and Benchmarks Track, 2023.

[89] J. Wang, S. Chen, and H. Li, "Towards generalizable deepfake detection with spatial-frequency collaborative learning and hierarchical cross-modal fusion," arXiv:2504.17223, 2025.

[90] M. Wang and W. Deng, "Deep visual domain adaptation: A survey," Neurocomputing, vol. 312, pp. 135–153, 2018.

[91] Y. Zhang, Y. Liu, and X. Chen, "Data-driven fairness generalization for deepfake detection," arXiv:2412.16428, 2024.

[92] W. Zhao, Y. Li, and Z. Sun, "DeepFake detection based on high-frequency enhancement network for highly compressed content," Expert Syst. Appl., vol. 249, art. 123732, 2024.

[93] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in Proc. IEEE Symp. Security and Privacy, 2017, pp. 39–57.

[94] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in Proc. ICLR, 2018; also available as arXiv:1706.06083.

[95] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in Proc. ICLR, 2015; also available as arXiv:1412.6572.

[96] M. Abbasi, P. Vaz, J. Silva, and P. Martins, "Comprehensive evaluation of deepfake detection models: Accuracy, generalization, and resilience to adversarial attacks," Applied Sciences, vol. 15, no. 3, 2025.

[97] Y. Li and S. Lyu, "Celeb-DF++: A large-scale challenging video deepfake benchmark for generalizable forensics," arXiv:2507.18015, 2025.

[98] K. Gupta, A. Joshi, and G. Kaushik, "A survey on multimedia-enabled deepfake detection: State-of-the-art tools and techniques, emerging trends, current challenges & limitations, and future directions," Discover Computing, 2025.

[99] G. Naskar, S. Mohiuddin, S. Malakar, E. Cuevas, and R. Sarkar, "Deepfake detection using deep feature stacking and meta-learning," Heliyon, vol. 10, no. 4, e25933, 2024.

[100] P. Zhou, X. Li, and T. Sim, "DeepfakeStack: A deep ensemble-based learning technique for deepfake detection," in Proc. 7th IEEE Int. Conf. on Cyber Security and Cloud Computing (CSCloud), 2020.

[101] L. Jiang, R. Li, W. Wu, C. Qian, and C. C. Loy, "DeeperForensics1.0: A large-scale dataset for real-world face forgery detection," in Proc. CVPR, 2020; also available as arXiv:2001.03024.

[102] A. Buslaev, V. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," arXiv:1809.06839, 2018.

[103] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in Proc. ICCV, 2017; also available as arXiv:1708.02002.

[104] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Proc. NeurIPS, 2017.

[105] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in Proc. NeurIPS, 2019.

[106] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, *et al.*, "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

Table 26: Training hyperparameters by stage. Values were selected through preliminary grid search and cross-validation on held-out splits.

Stage 1: MobileNetV4-Hybrid-Medium	
Hyperparameter	Value
Base architecture	MobileNetV4-Hybrid-Medium (timm)
Input resolution	256×256 RGB
Batch size	128
Optimizer	AdamW
Learning rate (initial)	1×10^{-4}
Weight decay	1×10^{-5}
LR scheduler	CosineAnnealingLR
Training epochs	50
Loss function	BCEWithLogitsLoss
Data augmentation	RandomHorizontalFlip, ColorJitter, RandomAffine, GaussianBlur
Dropout (final layers)	0.2
Gradient clipping	Max norm 1.0
Mixed precision training	Disabled (FP32)
Early stopping patience	10 epochs (validation AUC)
Stage 2: EfficientNetV2-B3 Expert	
Hyperparameter	Value
Base architecture	EfficientNetV2-B3 (timm)
Input resolution	256×256 RGB
Batch size	28
Optimizer	AdamW
Learning rate (initial)	5×10^{-5}
Weight decay	1×10^{-4}
LR scheduler	CosineAnnealingWarmRestarts (T_0=10)
Training epochs	50
Warmup epochs	— (not used in released script)
Loss function	Focal Loss (γ=2.0, α=0.25)
Data augmentation	RandAugment, Mixup (α=0.2), CutMix (α=1.0)
Dropout	0.3
Stochastic depth	0.2
Label smoothing	0.1
Gradient accumulation steps	1 (no accumulation)
Stage 3: LightGBM Meta-Model (optional)	
Hyperparameter	Value
Boosting type	gbdt (gradient boosting decision tree)
Objective	binary (binary logloss)
Metric	AUC
Number of leaves	64
Max depth	-1 (no limit)
Learning rate	0.05
Number of iterations	500
Feature fraction	0.8
Bagging fraction	0.9
Bagging frequency	5
Min data in leaf	50
Lambda L1	0.0
Lambda L2	0.1
Early stopping rounds	50 (validation AUC)
Verbosity	-1

Table 27: Per-generator performance breakdown on FaceForensics++ test split. Stage 2 EfficientNetV2-B3 calibrated model. Results show that GAN-based methods (Deepfakes, FaceSwap) are easier to detect than neural rendering techniques (NeuralTextures).

Generator Method	AUC ↑	F1 ↑	FNR ↓	FPR ↓
Deepfakes (DF)	0.9821	0.9234	0.0521	0.0312
Face2Face (F2F)	0.9713	0.9012	0.0689	0.0421
FaceSwap (FS)	0.9798	0.9187	0.0543	0.0334
NeuralTextures (NT)	0.9456	0.8543	0.1123	0.0623
Real (pristine)	—	—	—	0.0421 (avg FPR)
Overall (FF++ combined)	0.9697	0.8994	0.0719	0.0427

Table 28: Per-dataset optimal Stage 1 thresholds at target recall = 0.95. Note that threshold values vary substantially across datasets, motivating the need for multi-dataset calibration and adaptive threshold tuning.

Dataset	Optimal Threshold	Precision	Recall	F1
CelebDF-v2	0.45	0.9312	0.9500	0.9405
FaceForensics++	0.62	0.9123	0.9500	0.9308
DFDC	0.53	0.8834	0.9500	0.9155
DeeperForensics-1.0	0.58	0.9045	0.9500	0.9267
Combined validation	0.55	0.9078	0.9500	0.9284

Table 29: Calibration transfer analysis. ECE measured on test sets when applying temperature learned on FaceForensics++ validation vs. dataset-specific temperature. Lower ECE indicates better calibration. Results show modest degradation (ECE +1–2%) when transferring calibration across datasets, supporting our multi-dataset calibration approach.

Test Dataset	ECE (FF++ temp) ↓	ECE (own temp) ↓
CelebDF-v2	1.43%	0.89%
FaceForensics++	0.89%	0.89%
DFDC	2.12%	1.34%
DeeperForensics-1.0	1.78%	1.21%
Combined validation	1.55%	1.12%

Table 30: Comparison of mobile export formats for Stage 1 and Stage 2 models. TorchScript provides a good balance of size and compatibility for PyTorch Mobile integration, while ONNX offers broader framework support with slightly larger artifacts.

Model	Export Format	File Size	Compatibility	Inference Mode
Stage 1 MobileNetV4	TorchScript (.pt)	37.3 MB	PyTorch Mobile	CPU/GPU
Stage 1 MobileNetV4	ONNX (.onnx)	41.2 MB	ONNX Runtime	CPU/GPU
Stage 1 (quantized INT8)	TorchScript (.pt)	10.1 MB	PyTorch Mobile (CPU)	CPU only
Stage 2 EfficientNetV2-B3	TorchScript (.pt)	49.6 MB	PyTorch Mobile	CPU/GPU
Stage 2 EfficientNetV2-B3	ONNX (.onnx)	54.3 MB	ONNX Runtime	CPU/GPU
Stage 2 (quantized INT8)	TorchScript (.pt)	13.4 MB	PyTorch Mobile (CPU)	CPU only

Table 31: Training time and computational requirements. All experiments were conducted on a machine with a single NVIDIA RTX 5090 GPU (32 GB VRAM) and an AMD EPYC 8224P 24-core CPU. Training times are approximate and vary based on dataset size and early stopping behavior.

Stage	Model	Training Samples	Wall-Clock Time	Peak GPU Memory
Stage 1	MobileNetV4-Hybrid-Medium	1.87M	8–10 hours	18 GB
Stage 2	EfficientNetV2-B3	1.87M	18–22 hours	28 GB
Stage 2	GenConViT (hybrid mode)	1.87M	45–55 hours	35 GB
Stage 3	LightGBM meta-model	280K (meta-dataset)	15–20 minutes	N/A (CPU)
Stage 4	Threshold grid search	280K (validation)	2–3 hours	12 GB
Stage 5	Robustness sweeps	56K (test subset)	4–6 hours	12 GB